

Strukture podataka i algoritmi

Stek, red, heš tabele

dr Danijel Čabarkapa

Akademija strukovnih studija Šabac

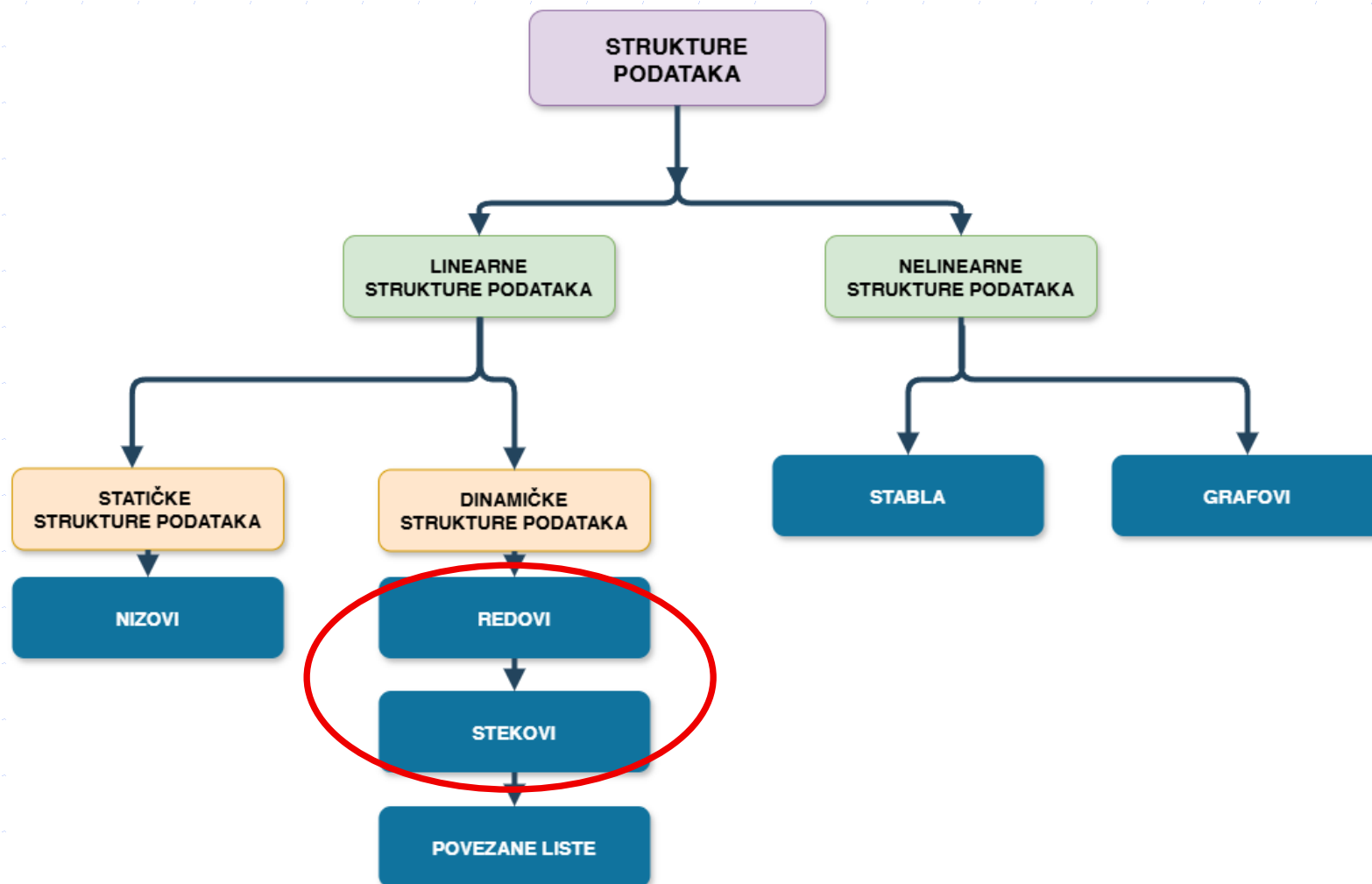
Odsek za medicinske i poslovno-tehnološke studije

Sadržaj:

1. Struktura podataka: Stek
2. Struktura podataka: Red
3. Struktura podataka: Heš tabele

1. Struktura podataka: Stek

Klasifikacija struktura podataka

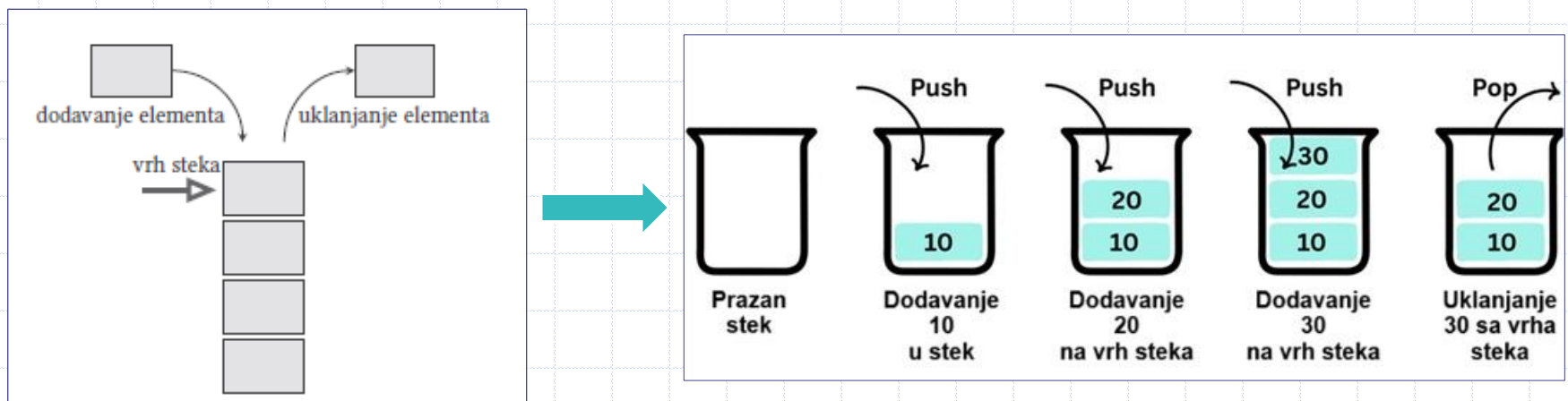


Stek struktura podataka

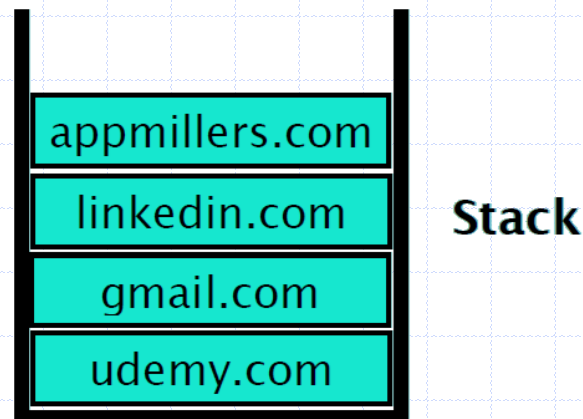
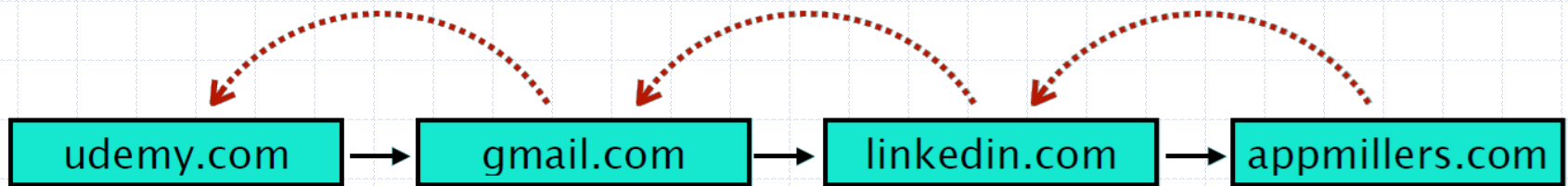
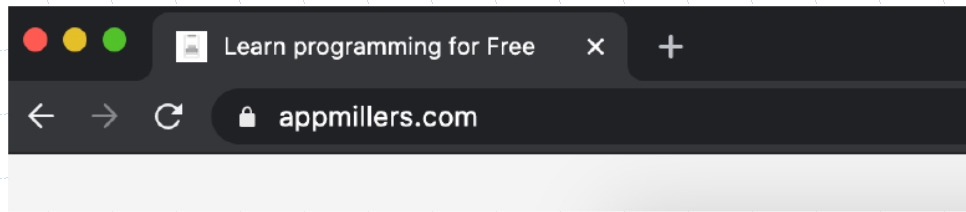
- **Stek** (*stack*) je linearna i dinamička struktura podataka koja funkcioniše po principu **LIFO** (*Last In First Out*)
- Stek je struktura podataka koja se, kao i povezana lista ili niz, sastoji od jednog niza podataka, ali kod koje **elemente možemo dodavati i uklanjati na samo jednom kraju**
- LIFO princip: poslednji dodati element u stek prvi je koji se može iz njega uzeti, tj. poslednji element koji je ušao, prvi je koji ide van
- Ovu struktura podataka karakterišu dve specifične operacije – **push** za dodavanje elemenata i **pop** za njihovo uklanjanje
- Stek se može implementirati na više načina: kao niz fiksne veličine, dinamički niz ili jednostruko/dvostruko povezana lista
- Stek teorijski ne podržava operacije arbitrarnog pretraživanja ili umetanja novih elemenata bilo gde u listi

Stek struktura podataka

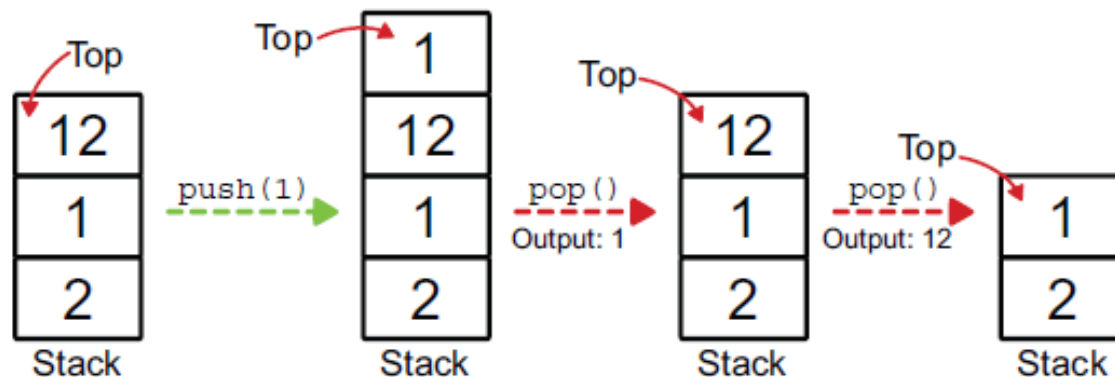
- Kod steka su elementi niza poređani „jedan na drugi“, pri čemu se sva dodavanja i uklanjanja elemenata steka izvršavaju samo na vrhu steka (*top of the stack*)
- Stek koristi dve specifične operacije – **push** za dodavanje elemenata i **pop** za njihovo uklanjanje
- Poredak elemenata po kojem se oni uklanjaju sa vrha steka je obrnut onom po kojem su oni dodati na vrh steka



Stek - primena

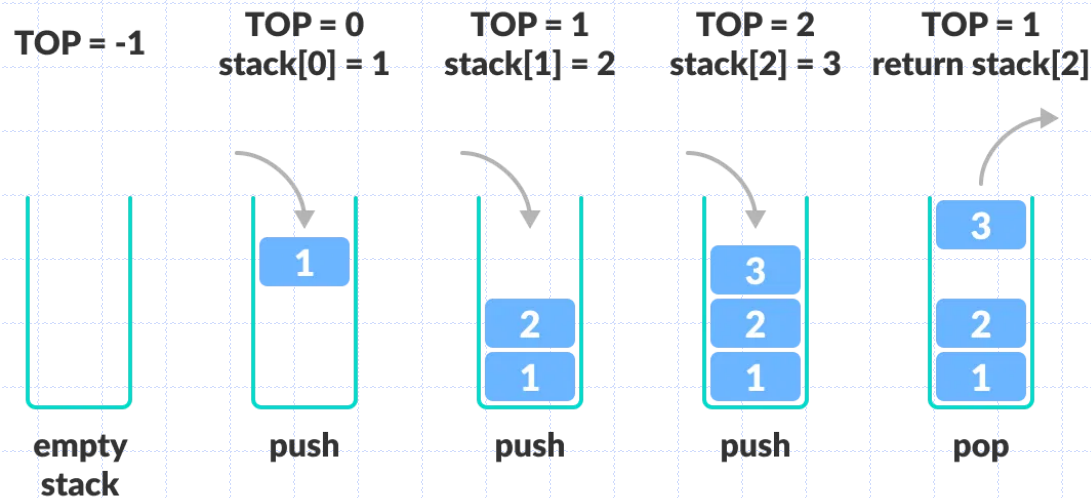
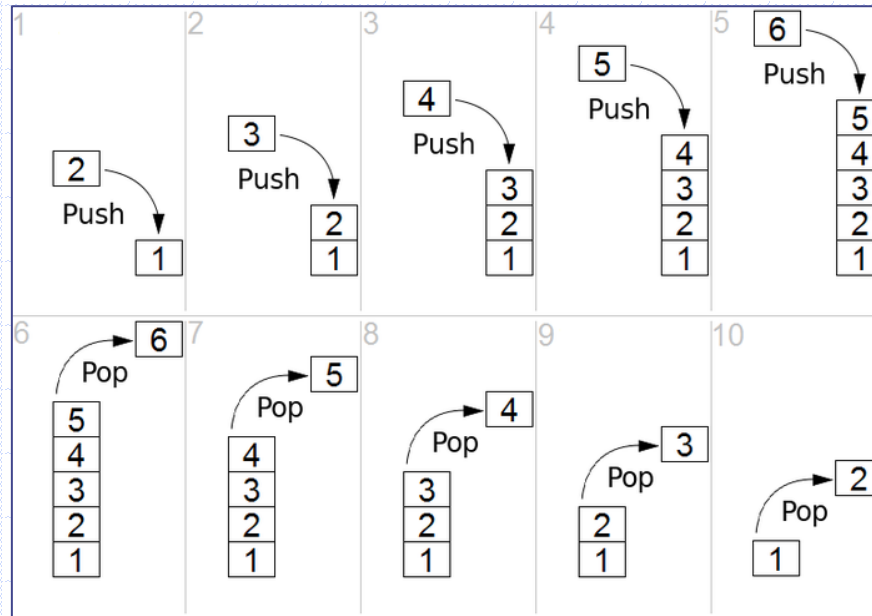


Stek struktura podataka



- U skladu sa ograničenjem LIFO principa, kod steka se održava redosled dodavanja novih elemenata i potrebno je pratiti poslednji element koji je dodan na stek, a koji se naziva vrh steka (**Top**)
- U praksi, formira se niz elemenata steka prema redosledu njihovog dodavanja, a vrh steka će biti strana niza gde dodajemo i uklanjamo elemente (bez obzira na to kako je niz sačuvan)

Stek struktura podataka



Memorija steka

`customStack = [1,2,3,4]`

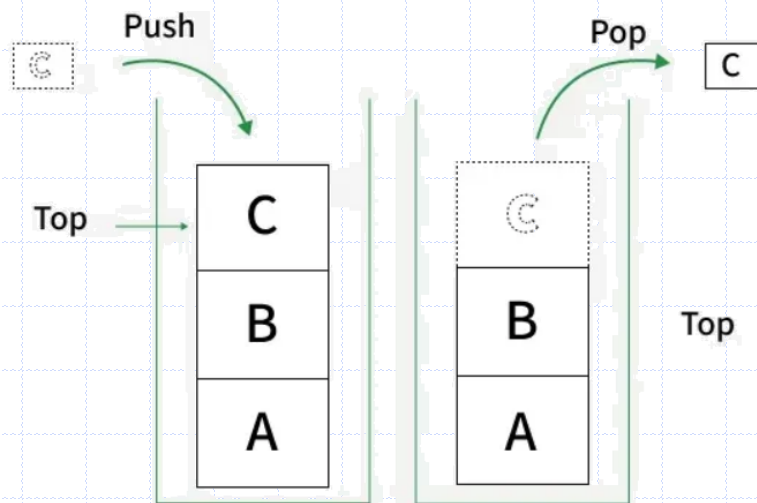
				4					
				3					
				2					
				1					

`customStack = [1,2]`

`customStack.pop()` → 2

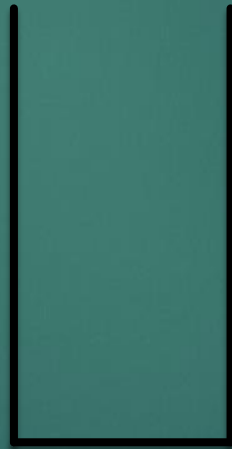
				2					
				1					

Osnovne operacije steka



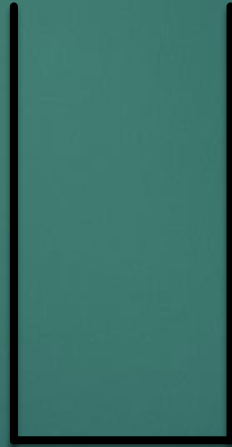
- Za stek se definišu sledeće operacije:
 - `push(c)`: dodavanje novog elementa na vrh steka
 - `pop()`: brisanje elementa sa vrha steka
 - `size()`: vraća broj elemenata steka
 - `isEmpty()`: proverava da li je stek prazan
 - `top()/peek()`: vraća vrednost elementa na vrhu, bez uklanjanja istog

PUSH operacija: unos novog elementa NA VRH
STEKA, jednostavna operacija sa $O(1)$



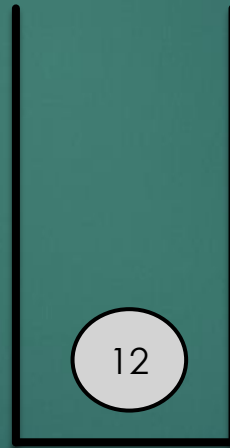
PUSH operacija: unos novog elementa NA VRH
STEKA, jednostavna operacija sa $O(1)$

```
stack.push(12)
```



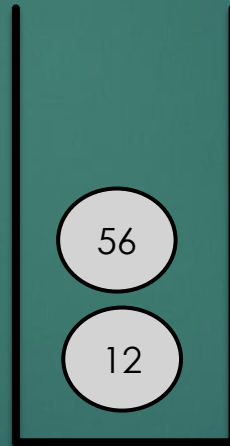
PUSH operacija: unos novog elementa NA VRH
STEKA, jednostavna operacija sa $O(1)$

`stack.push(56)`



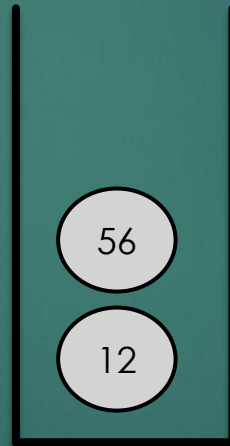
PUSH operacija: unos novog elementa NA VRH
STEKA, jednostavna operacija sa $O(1)$

`stack.push(56)`



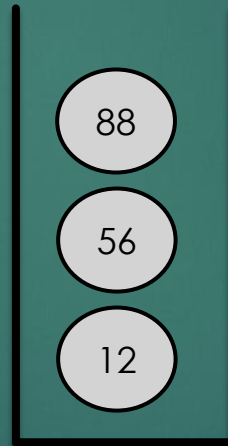
PUSH operacija: unos novog elementa NA VRH
STEKA, jednostavna operacija sa $O(1)$

`stack.push(88)`



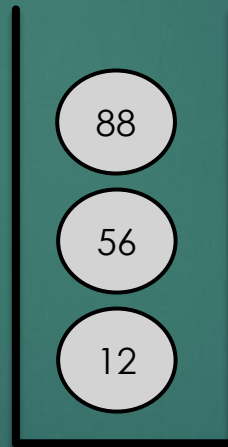
PUSH operacija: unos novog elementa NA VRH
STEKA, jednostavna operacija sa $O(1)$

`stack.push(88)`

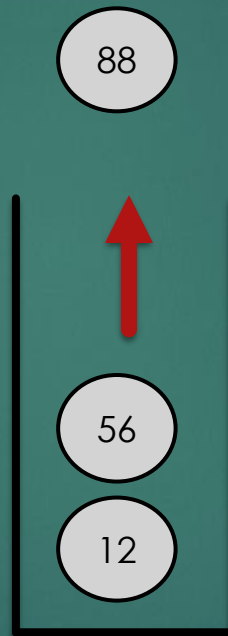


POP operacija: uklanja se poslednji uneti element
SA VRHA STEKA, jednostavna LIFO operacija sa $O(1)$

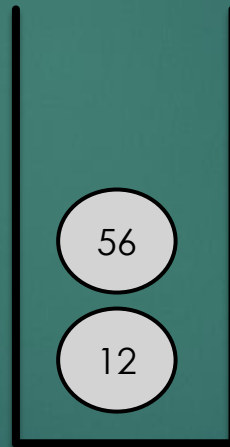
`stack.pop()`



POP operacija: uklanja se poslednji uneti element
SA VRHA STEKA, jednostavna LIFO operacija sa $O(1)$

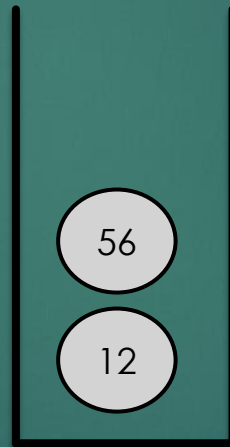


POP operacija: uklanja se poslednji uneti element
SA VRHA STEKA, jednostavna LIFO operacija sa $O(1)$



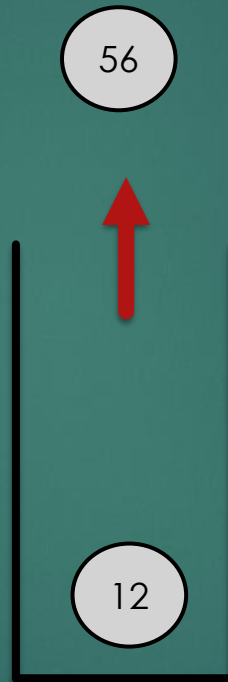
POP operacija: uklanja se poslednji uneti element
SA VRHA STEKA, jednostavna LIFO operacija sa $O(1)$

`stack.pop()`

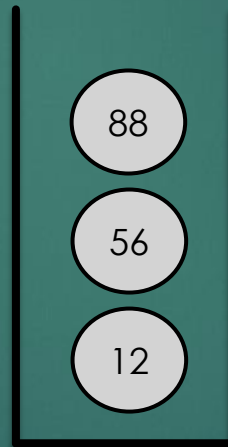


POP operacija: uklanja se poslednji uneti element
SA VRHA STEKA, jednostavna LIFO operacija sa $O(1)$

`stack.pop()`

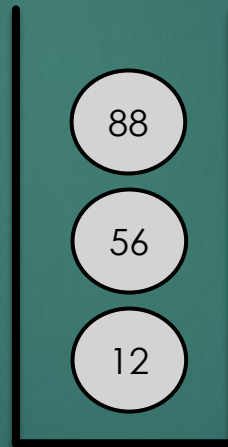


PEEK operacija: vraća element SA VRHA STEKA, bez njegovog uklanjanja, jednostavna LIFO operacija sa $O(1)$



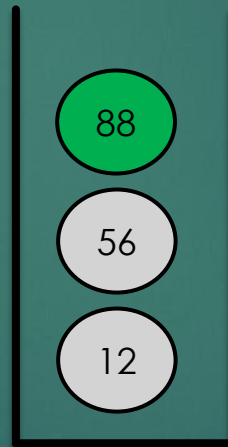
PEEK operacija: vraća element SA VRHA STEKA, bez njegovog uklanjanja, jednostavna LIFO operacija sa $O(1)$

`stack.peek()`



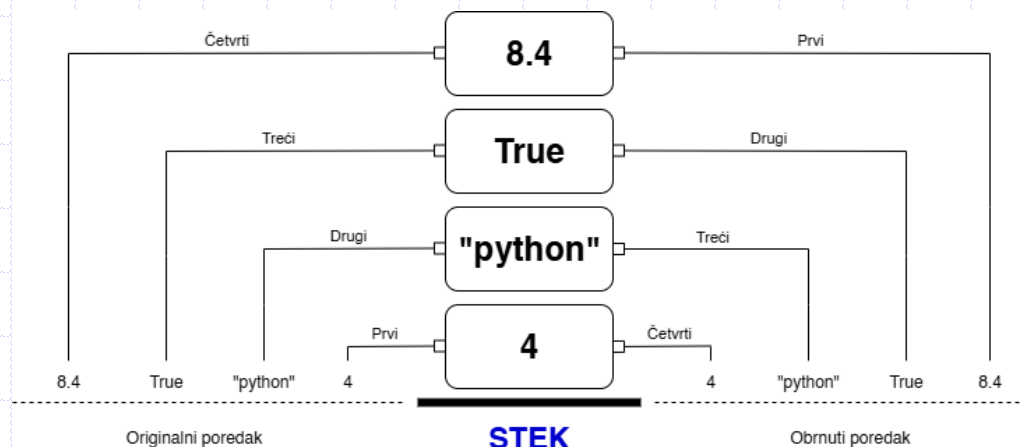
PEEK operacija: vraća element SA VRHA STEKA, bez njegovog uklanjanja, jednostavna LIFO operacija sa $O(1)$

`stack.peek()`



`peek()` metoda će vratiti 88, ali će struktura steka ostati nepromenjena !!!

Osnovne operacije steka



Stek operacija	Sadržaj steka	Povratna vrednost
s.is_empty()	[]	True
s.push(4)	[4]	
s.push('python')	[4, 'python']	
s.peek()	[4, 'python']	'python'
s.push(True)	[4, 'python', True]	
s.size()	[4, 'python', True]	3
s.is_empty()	[4, 'python', True]	False
s.push(8.4)	[4, 'python', True, 8.4]	
s.pop()	[4, 'python', True]	8.4
s.pop()	[4, 'python']	True
s.size()	[4, 'python']	2

Osnovne operacije steka

- Python liste obezbeđuju sopstvene ugrađene (built-in) metode koje se mogu direktno koristiti za rad sa stekom
 - `append()` metodom se dodaje element na kraj liste
 - `pop()` metoda vraća vrednost elementa na vrhu liste (steka)

```
1  stack = []
2
3  # append() funkcija za unos elementa u stek
4  stack.append('a')
5  stack.append('b')
6  stack.append('c')
7
8  print('Početni stek:')
9  print(stack)
10 # pop() funkcija uklanja element po LIFO principu
11 print('\nElementi uklonjeni iz steka:')
12 print(stack.pop())
13 print(stack.pop())
14 print(stack.pop())
15
16 print('\nStek nakon uklonjenih elemenata:')
17 print(stack)
18
19 # iapisom print(stack.pop()) javlja IndexError
20 # grešku jer stek nije prazan
```

Početni stek:
['a', 'b', 'c']

Elementi uklonjeni iz steka:
c
b
a

Stek nakon uklonjenih elemenata:
[]

Osnovne operacije steka

- Metode za rad sa stekom definisanim putem klase **myStack**:

```
class myStack:
    def __init__(self):
        self.container = []

    def isEmpty(self):
        return self.container.size() == 0

    def push(self, item):
        self.container.append(item)

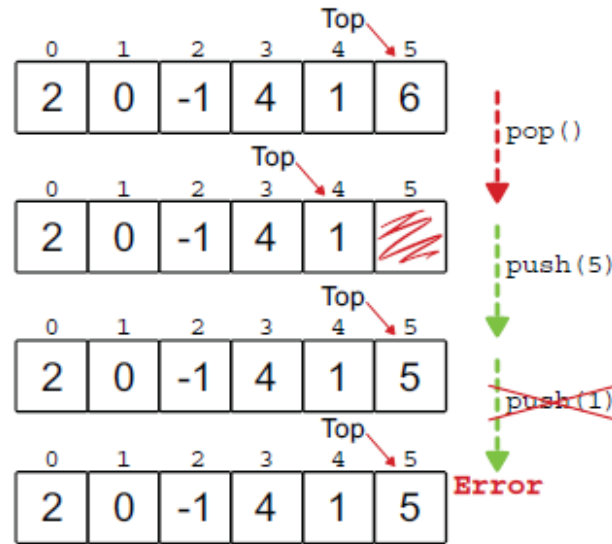
    def pop(self):
        return self.container.pop()

    def size(self):
        return len(self.container)
```

Stek – implementacija

- Python ne poseduje ugrađeni tip podatka tipa stek, pa se implementacija steka vrši upotrebom drugih struktura podataka
- Stek se može implementirati na sledeće načine:
 - 1) preko nizova
 - stek fiksne dužine (statički niz)
 - stek promenljive dužine (dinamički niz)
 - 2) preko povezanih listi
- Implementacija steka preko povezanih listi nije jednostavno primenljiva, ali je efikasna
- Kod implementacije steka putem statičkog niza (stek ima fiksnu veličinu), važno je proveriti da li je stek pun ili ne, jer će pokušaj unošenja elementa u pun stek generisati grešku prekoračenja (*overflow*)

Stek kao statički niz



- Ako se koristi statički niz za čuvanje elemenata steka, moguće je dodavanje novih elemenata na kraj niza ili njihovo uklanjanje sa kraja
- Obe ove operacije su sa $O(1)$ vremenskom složnošću i ne zahtevaju nikakvu dodatnu memoriju
- Problem kod statičkih nizova je njihova fiksna veličina, pa stek koji se formira na ovaj način zahteva podešavanje njegovog maksimalnog kapaciteta, koji nije moguće naknadno promeniti

Stek kao statički niz

- Stek se može implementirati pomoću nizova upotrebom:
 - Celobrojnog niza za čuvanje elemenata
 - Promenljive `capacity` za predstavljanje max. veličine steka
 - Promenljive `top` za praćenje indeksa vrha steka, gde `top = -1` označava prazan stek

```
1  class myStack:
2
3      def __init__(self, cap):
4
5          # arr je niz za čuvanje elemenata
6          self.arr = [0] * cap
7
8          # max veličina steka
9          self.capacity = cap
10
11         # index top elementa
12         self.top = -1
```

Definisanje klase `myStack` i dodavanje kontejnera za čuvanje elemenata steka, sa konstruktorom `__init__` se deklariše prazan stek

Inicijalizacija praznog niza dužine `cap` koji će predstavljati stek

Stek kao statički niz

- **Dodavanje elemenata u stek `push()`**, ako je stek popunjen, javlja se prekoračenje (Overflow condition)
 - Pre unosa novog elementa u stek proverava se da li je stek pun
 - Ako jeste, (`top==capacity-1`), nastalo je prekoračenje i ne može se uneti novi element u stek, u suprotnom, vrednost `top` se povećava za 1 (`top=top+1`) i nova vrednost se ubacuje na vrh steka
 - Elementi se unose u stek dok se ne dostigne vrednost `cap`

```
1  def push(self, x):
2
3      if self.top == self.capacity - 1:
4          print("Stack Overflow")
5          return
6
7      self.top += 1
8
9      self.arr[self.top] = x
```

- Vremenska složenost za `push()` steka je $O(1)$
- Prostorna složenost za `push()` steka je $O(1)$

Stek kao statički niz

Dodavanje elemenata u stek push()

01 Step Empty Stack

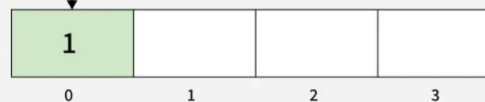
Capacity = 4

top = -1



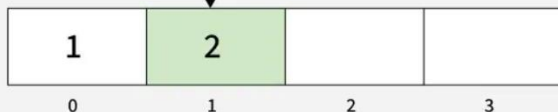
02 Step Push Element 1 Into Stack

top = 0



03 Step Push Element 2 Into Stack

top = 1



04 Step Push Element 3 Into Stack

Capacity = 4

top = 2

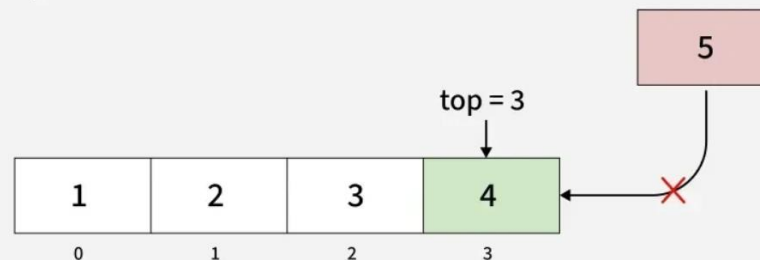


05 Step Push Element 4 Into Stack

top = 3



06 Step Push Element 5 Into Stack (Stack Overflow)



Stek kao statički niz

- **Uklanjanje elemenata iz steka pop()**, elementi se uklanjaju obrnutim redosledom od unosa, ako je stek prazan, onda nastaje uslov Underflow
 - Pre uklanjanja elementa steka proverava se da li je stek prazan
 - Ako jeste, (`top == -1`), nastao je **Underflow** i ne može se ukloniti ni jedan element iz steka
 - U suprotnom, sačuvamo vrednost `top`, smanjimo je za 1 (`top = top - 1`) i vraćamo sačuvanu vrednost za `top`

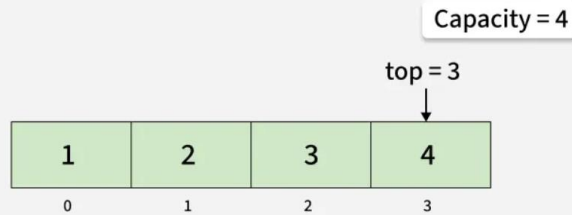
```
1  def pop(self):
2
3      if self.top == -1:
4          print("Stack Underflow")
5          return -1
6
7      value = self.arr[self.top]
8      self.top -= 1
9
10     return value
```

- Vremenska složenost za `pop()` steka je $O(1)$
- Prostorna složenost za `pop()` steka je $O(1)$

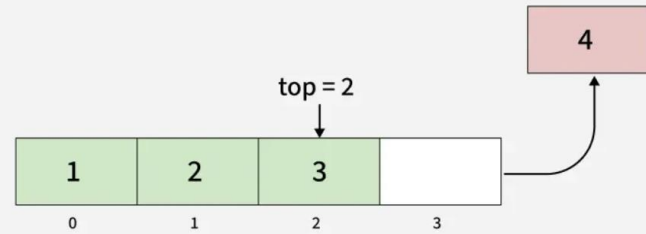
Stek kao statički niz

Uklanjanje elemenata iz steka pop()

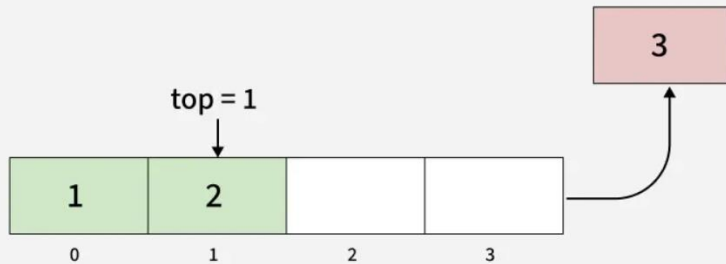
01 Step Initial Stack



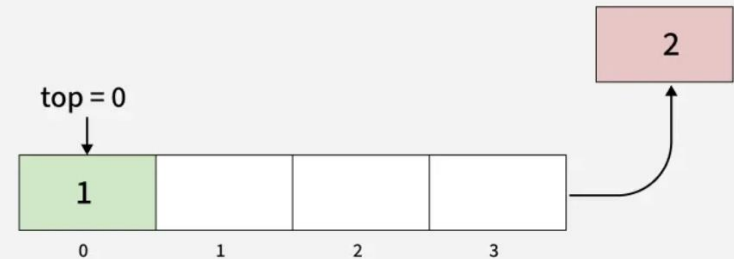
02 Step Pop Element 4 From Stack



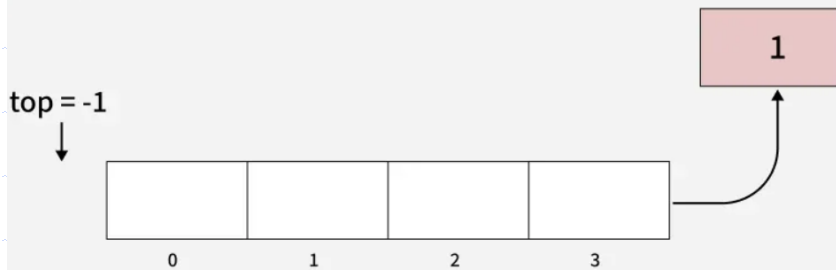
03 Step Pop Element 3 From Stack



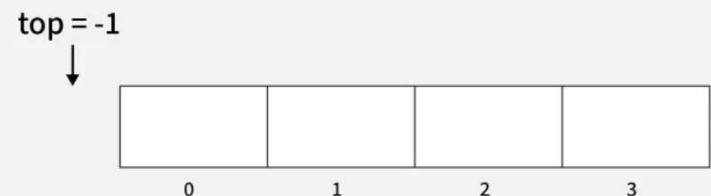
04 Step Pop Element 2 From Stack



05 Step Pop Element 1 From Stack



06 Step Empty Stack (Popping an Element will cause underflow)



Stek kao statički niz

- Operacija **peek()**, vraća element sa vrha steka, ako je stek prazan (**top == -1**), štampa se "Stek je prazan", u suprotnom se vraća indeks **top** elementa
- Operacija **isEmpty()**, vraća **True** ako je stek prazan, u suprotnom **False**, proverava se vrednost **top** elementa, ako je (**top == -1**) stek je prazan i vraća se **True**
- Operacija **isFull()**, vraća **True** ako je stek popunjen, u suprotnom **False**, proverava se vrednost **top** elementa, ako je (**top == capacity - 1**) stek je popunjen i vraća se **True**, u suprotnom **False**

```
1 def peek(self):
2
3     if self.top == -1:
4         print("Stack is Empty")
5         return -1
6
7     return self.arr[self.top]
```

```
1 def isEmpty(self):
2
3     return self.top == -1

1 def isFull(self):
2     return self.top == self.capacity - 1
```

- Vremenska složenost za **peek()**, **isEmpty()**, **isFull()** je **O(1)**
- Prostorna složenost za **peek()**, **isEmpty()**, **isFull()** je **O(1)**

Stek kao statički niz

```
1 class myStack:
2     # konstruktor
3     def __init__(self, cap):
4         self.capacity = cap
5         self.arr = [0] * self.capacity
6         self.top = -1
7
8     # push operacija
9     def push(self, x):
10        if self.top == self.capacity - 1:
11            print("Stek Overflow")
12            return
13        self.top += 1
14        self.arr[self.top] = x
15
16    # pop operacija
17    def pop(self):
18        if self.top == -1:
19            print("Stek Underflow")
20            return -1
21        val = self.arr[self.top]
22        self.top -= 1
23        return val
24
25    # peek (top) operacija
26    def peek(self):
27        if self.top == -1:
28            print("Stek je prazan")
29            return -1
30        return self.arr[self.top]
31
32    # provera da li je stek prazan
33    def isEmpty(self):
34        return self.top == -1
35
36    # provera da li je stek pun
37    def isFull(self):
38        return self.top == self.capacity - 1
39
40 if __name__ == "__main__":
41     st = myStack(4)
42     # push elementi
43     st.push(1)
44     st.push(2)
45     st.push(3)
46     st.push(4)
47
48     # uklanjanje jednog elementa
49     print("Uklonjen:", st.pop())
50
51     # provera top elementa
52     print("Top element:", st.peek())
53
54     # provera da li stek prazan
55     print("Da li je stek prazan: ", "DA" if st.isEmpty() else "NE")
56
57     # provera da li stek pun
58     print("Da li je stek pun: ", "DA" if st.isFull() else "NE")
59
60 Uklonjen: 4
61 Top element: 3
62 Da li je stek prazan: NE
63 Da li je stek pun: NE
```

Stek kao dinamički niz - primer

- Kada se koristi niz fiksne veličine, stek ima max. dužinu koja ne može da se menja; Da bi se to prevazišlo koriste se dinamički nizovi koji automatski menjaju dužinu kako se elementi dodaju ili uklanjaju, što čini stek fleksibilnijim

```
1 class myStack:
2     def __init__(self):
3         self.arr = []
4
5     # push operacija
6     def push(self, x):
7         self.arr.append(x)
8
9     # pop operacija
10    def pop(self):
11        if not self.arr:
12            print("Stack Underflow")
13            return -1
14        return self.arr.pop()
15
16    # peek operacija
17    def peek(self):
18        if not self.arr:
19            print("Stack is Empty")
20            return -1
21        return self.arr[-1]
22
23    # provera da li je prazan stek
24    def isEmpty(self):
25        return len(self.arr) == 0
26
27    # tekuća veličina steka
28    def size(self):
29        return len(self.arr)
30
```

```
31
32 if __name__ == "__main__":
33     st = myStack()
34
35     # unošenje elemenata
36     st.push(1)
37     st.push(2)
38     st.push(3)
39     st.push(4)
40
41     # uklanjanje jednog elementa
42     print("Uklonjen:", st.pop())
43
44     # provera top elementa
45     print("Top element:", st.peek())
46
47     # provera prazan stek
48     print("Da li je stek prazan:", "DA" if st.isEmpty() else "NE")
49
50     # provera tekuće veličine steka
51     print("Tekuća veličina steka:", st.size())
```

```
Uklonjen: 4
Top element: 3
Da li je stek prazan: NE
Tekuća veličina steka: 3
```

Fixed size stack vs. Dynamic size stack

	Stek sa statičkim nizom	Stek sa dinamičkim nizom
Alokacija memorije	Fiksna, određena definisanjem steka	Menja se automatski sa promenom elemenata
Fleksibilnost	Ograničena, veličina steka ne može biti veća od predefinisane	Potpuno fleksibilan, nema predefinisnog ograničenja veličine steka
Performanse	Neznatno brži jer ne postoji prekoračenje veličine	„Cena“ neograničene veličine steka
Lakoća upotrebe	Potrebno podešavati uslove underflow/overflow	Mnogo pogodniji, nema overflow uslova
Use case	Kada je maksimalna veličina steka unapred poznata	Kada su zahtevi veličine steka nepoznati ili promenljivi

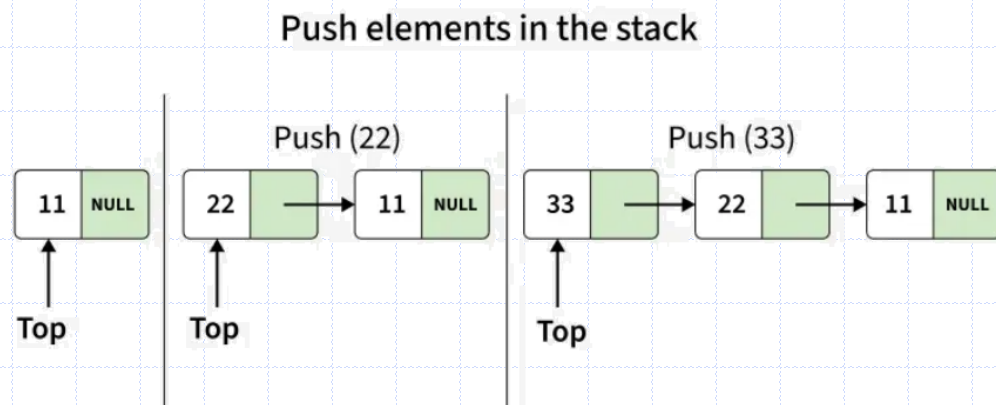
Stek sa povezanim listama

- Stek se može implementirati pomoću povezane liste upotrebom:
 - Strukture čvora/kalse koja sadrži:
 - ▶ `data` → za čuvanje elementa steka
 - ▶ `next` → pokazivač/referenca na sledeći čvor u steku
 - Pointera/reference ili pokazivača trenutnog top čvora steka
 - Inicijalno, `top=null` predstavlja prazan stek

```
1  # Struktura čvora
2  class Node:
3      def __init__(self, x):
4          self.data = x
5          self.next = None
6
7
8  # Klasa steka
9  class myStack:
10     def __init__(self):
11
12         # inicijalno je stek prazan
13         self.top = None
```

Stek sa povezanim listama

- **Dodavanje elemenata u stek push()**, za razliku od steka sa nizom, kod povezane liste nema fiksne veličine steka, Overflow se javlja samo kada se memorija iscrpi:
 - Kreira se novi čvor sa datom vrednošću
 - Next pokazivač novog čvora se postavlja na trekući top element steka
 - Top pointer se ažurira kako bi pokazivao na novi čvor

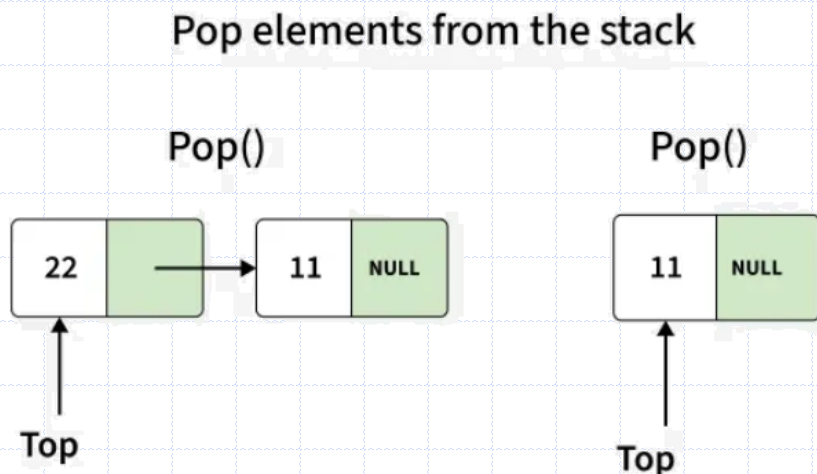


```
1 def push(self, x):
2     temp = Node(x)
3     temp.next = self.top
4     self.top = temp
```

- Vremenska složenost za **push()** steka je **$O(1)$**
- Prostorna složenost za **push()** steka je **$O(1)$**

Stek sa povezanim listama

- **Uklanjanje elemenata iz steka pop()**, uklanje top element iz steka, ako je stek prazan, postoji Underflow uslov:
 - Pre brisanja proverava se da li je stek prazan (`top == NULL`).
 - Ako je stek prazan, dešava se Underflow uslov i brisanje nije moguće
 - U suprotnom, tekući top čvor se čuva u privremenom pokazivaču
 - Prebaciti top pointer na sledeći čvor
 - obrisati privremeni čvor radi oslobađanja memorije



```
1 def pop(self):
2
3     if self.top is None:
4         print("Stack Underflow")
5         return -1
6
7     temp = self.top
8     self.top = self.top.next
9     val = temp.data
10
11     del temp
12     return val
```

- Vremenska složenost za `push()` steka je $O(1)$
- Prostorna složenost za `push()` steka je $O(1)$

Stek sa povezanim listama - primer

```
1 # Struktura čvora
2 class Node:
3     def __init__(self, x):
4         self.data = x
5         self.next = None
6 # Implementacija steka povezanom listom
7 class myStack:
8     def __init__(self):
9         # inicijalno je stek prazan
10        self.top = None
11        self.count = 0
12
13    # push operacija
14    def push(self, x):
15        temp = Node(x)
16        temp.next = self.top
17        self.top = temp
18        self.count += 1
19
20    # pop operacija
21    def pop(self):
22        if self.top is None:
23            print("Stack Underflow")
24            return -1
25        temp = self.top
26        self.top = self.top.next
27        val = temp.data
28
29        self.count -= 1
30        return val
31
```

```
31
32    # peek operacija
33    def peek(self):
34        if self.top is None:
35            print("Stek je prazan")
36            return -1
37        return self.top.data
38
39    # provera da li je stek prazan
40    def isEmpty(self):
41        return self.top is None
42
43    # dužina steka
44    def size(self):
45        return self.count
46
47 if __name__ == "__main__":
48     st = myStack()
49
50     # elementi koji se dodaju
51     st.push(1)
52     st.push(2)
53     st.push(3)
54     st.push(4)
55
56     # uklanjanje jednog elementa
57     print("Uklonjen:", st.pop())
58
59     # provera top elementa
60     print("Top element:", st.peek())
61
62     # provera da li je stek prazan
63     print("Da li je stek prazan:", "DA" if st.isEmpty() else "NE")
64
65     # provera veličine tekućeg steka
66     print("DUžina steka je:", st.size())
```

Uklonjen: 4
Top element: 3
Da li je stek prazan: NE
DUžina steka je: 3

Stek – primene

- **Pozivi funkcija:** Stek se koristi za praćenje poziva funkcija, omogućavajući programu da se vrati na ispravnu lokaciju nakon što je funkcija završila sa izvršavanjem
- **Rekurzija:** Stek se koristi za čuvanje lokalnih promenljivih i povratnih adresa rekurzivnih poziva funkcija, omogućavajući programu da prati trenutno stanje rekurzije
- **Evaluacija izraza:** Stek se koristi za evaluaciju izraza u postfiksnoj notaciji (Reverse Polish Notation)
- **Parsiranje sintakse:** Stek se koristi za proveru validnosti sintakse u programskim jezicima i drugim formalnim jezicima
- **Upravljanje memorijom:** Stek se koristi za dodeljivanje i upravljanje memorijom u nekim operativnim sistemima i programskim jezicima
- Koristi se za rešavanje „popularnih“ problema kao što su: Next Greater, Previous Greater, Next Smaller, Previous Smaller, Largest Area in a Histogram and Stock Span Problems

Stek – prednosti i nedostaci

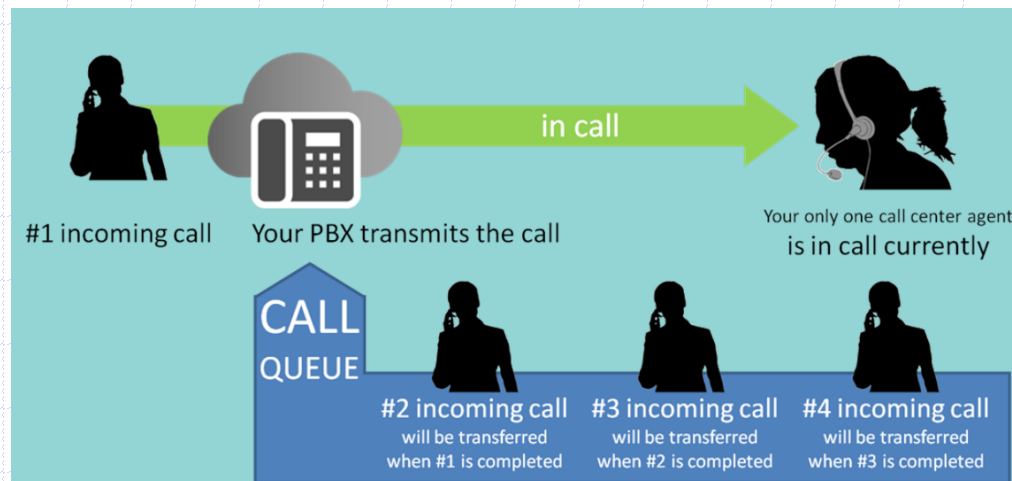
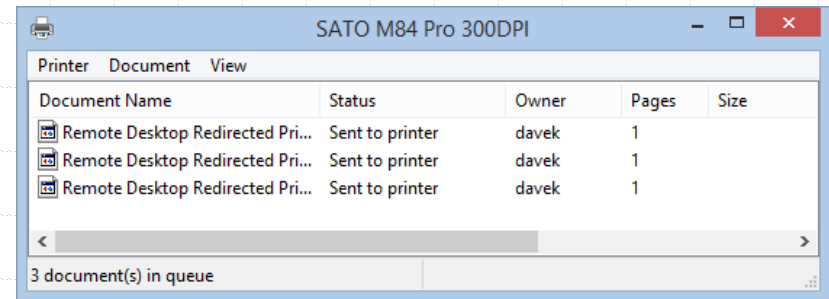
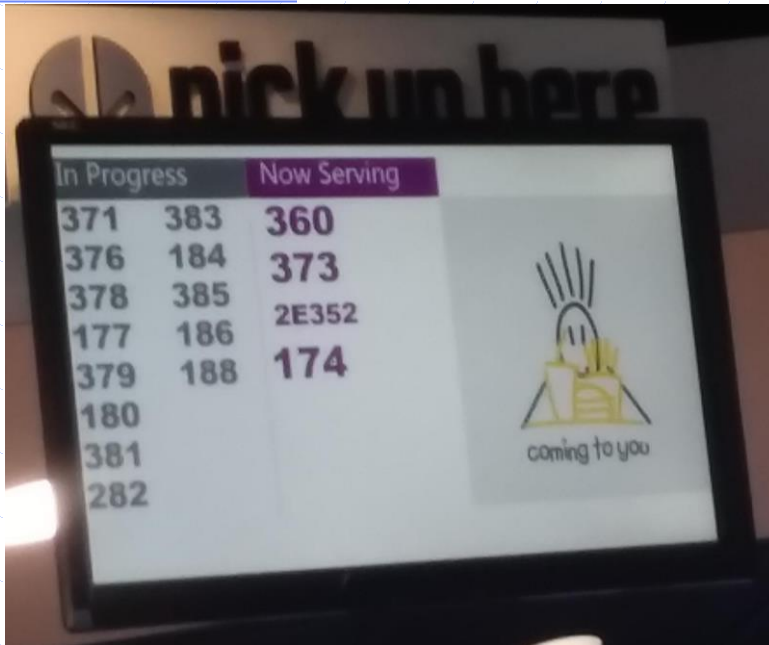
- Prednosti steka:
 - Jednostavnost - stek je jednostavna i lako razumljiva struktura podataka, primenljiva za širok opseg zadataka
 - Efikasnost - **push** i **pop** operacije imaju const. vremensku složenost $O(1)$, pa je pristup podacima steka efikasan
 - LIFO princip: efikasan i koristan u programiranju, od poziva funkcija, preko izračunavanja izraza ...
 - Ograničeno korišćenje memorije: stek samo čuva elemente koji se dodaju komandom **push**, pa je memorijski efikasniji od drugih struktura
- Nedstaci steka:
 - Ograničen pristup: pristup elementima steka preko **top** člana, problem pristupa i izmene elemenata u sredini
 - Nema mogućnost slučajnog pristupa elementima
 - Ograničen kapacitet steka: stek ima ograničen kapacitet, pa se ne može upotrebiti za slučaj čuvanja većeg ili nepoznatog broja elemenata
 - Prekoračenje (overflow): ako se unese veći broj elemenata nego što stek podržava, javlja se prekoračenje i mogući gubitak podataka

2. Struktura podataka: Red

Red

- **Red** (*queue*) ili „red za čekanje“ je linearna i dinamička struktura podataka koja funkcioniše po principu **FIFO** (*First In First Out*)
- Red je struktura podataka koja se, kao i povezana lista ili niz, sastoji od niza podataka, kod koje **elemente možemo dodavati na jednom kraju, a uklanjati ih na drugom**
- **FIFO princip**: Prvi dodati element u red prvi je koji se može iz njega ukloniti, dodavanje elemenata je moguće u bilo kom trenutku, ali je moguće ukloniti samo onaj element koji je najduže u redu, elementi reda se dodaju na kraj, a uklanjaju se sa početka reda
- Red na šalteru je primer strukture red, osoba koja prva stigne, prva će završiti posao na šalteru, dok će osobe koje su kasnije došle dolaziti na kraj reda i čekati...

Redovi - primene



Redovi - primene

PRIMENA REDOVA



CPU Scheduling

CPU zadaci su postavljeni u red i izvršavaju se jedan za drugim



Job Scheduling

Štampači štampaju višestruke dokumente koji su u formi reda čekanja



Networking

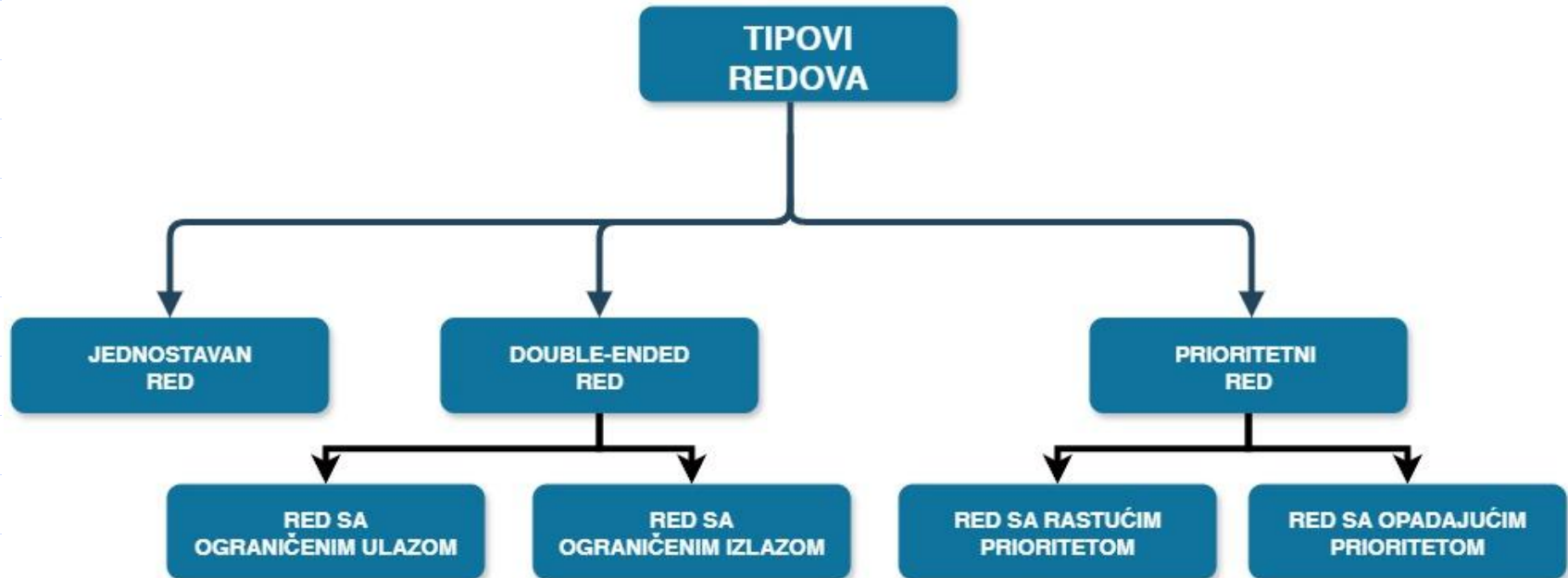
Ruteri, svičevi i mrežni uređaji obrađuju podatke koji su u formi redova



Buffer Management

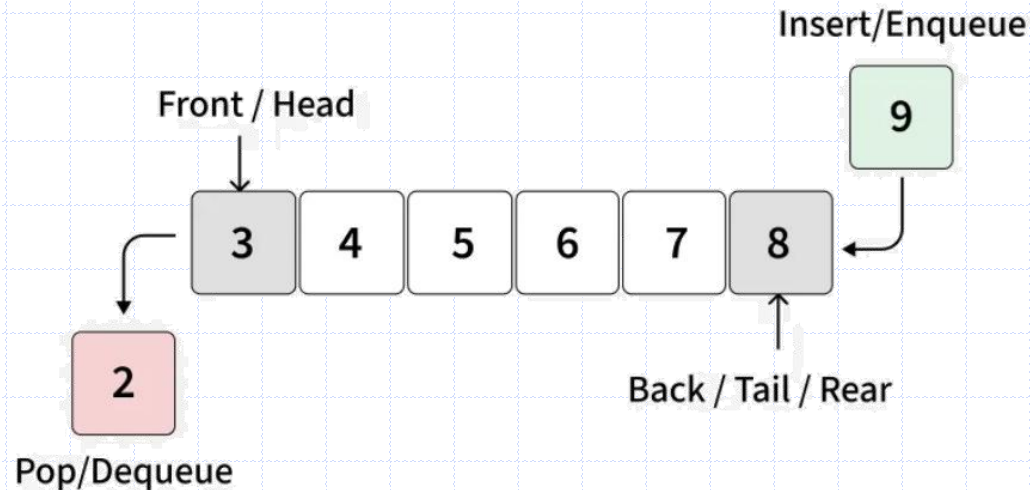
Koriste se kao buffer memorija između sporih i brzih uređaja (keyboard-CPU network devices)

Vrste redova



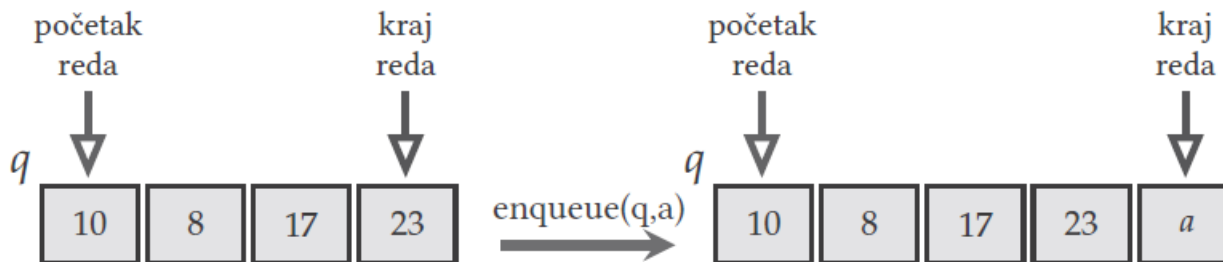
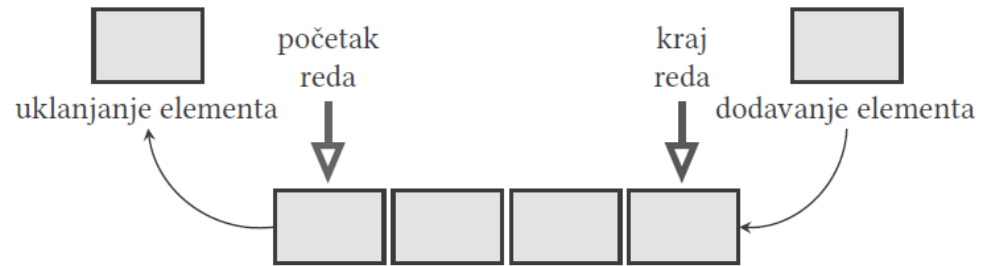
- **Jednostavan red**: FIFO princip (unos samo na kraju-rear, brisanje samo na početku-front, kreira se kroz povezane ili kružne liste)
- Red sa dva kraja ili **double-ended** (*deque*): unos ili brisanje može da se vrši na oba kraja
- **Red sa prioritetom**: svakom elementu dodeljen je prioritet, a brisanje se uvek dešava na osnovu prioriteta a ne samo pozicije elementa

Red - terminologija



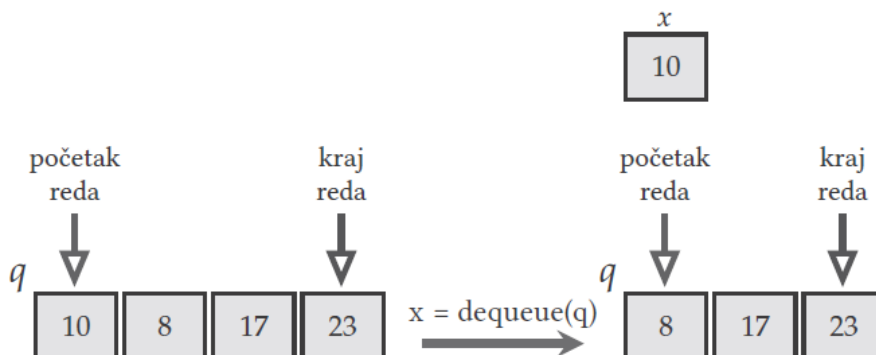
- Za redove su definišu dve ključne operacije:
 - a) dodavanje elemenata na kraj reda **Enqueue**
 - b) Uklanjanje elemenata sa početka reda **Dequeue**
- **Front/Head**: Pozicija prvog dodatog elementa u redu koji će biti procesiran i prvi koji će biti uklonjen iz reda
- **Rear/Tail/Back**: Pozicija poslednjeg unetog elementa u red
- **Size**: Predstavlja tekući broj elemenata reda
- **Capacity**: Maksimalni broj elemenata koji mogu da se čuvaju u redu

Red



Enqueue

(dodavanje elementa **a** na kraj reda **q** koji sadrži elemente 10,8,17 i 23)



Dequeue

(uklanjanje prvog elementa **10** sa početka reda **q** koji sadrži elemente 10,8,17 i 23)

Enqueue operacija: dodavanje novog elementa
NA KRAJ REDA





Enqueue operacija: dodavanje novog elementa
NA KRAJ REDA

```
queue.enqueue(10)
```



Enqueue operacija: dodavanje novog elementa
NA KRAJ REDA

```
queue.enqueue(10)
```



Enqueue operacija: dodavanje novog elementa
NA KRAJ REDA

```
queue.enqueue(4)
```



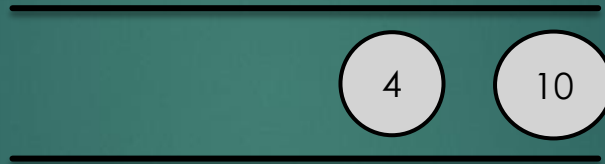
Enqueue operacija: dodavanje novog elementa
NA KRAJ REDA

```
queue.enqueue(4)
```



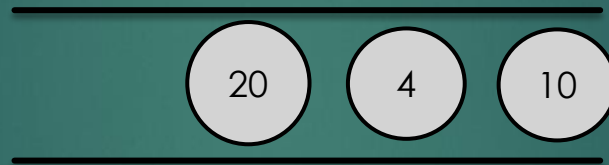
Enqueue operacija: dodavanje novog elementa
NA KRAJ REDA

```
queue.enqueue(20)
```

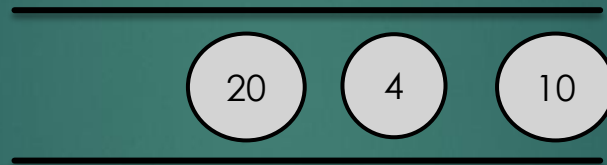


Enqueue operacija: dodavanje novog elementa
NA KRAJ REDA

```
queue.enqueue(20)
```

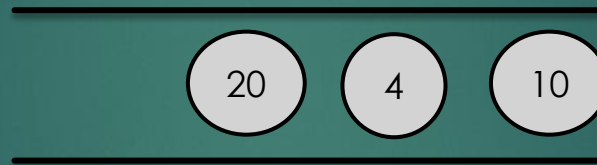


Deque operation: uklanjanje elementa SA
POČETKA REDA // FIFO struktura



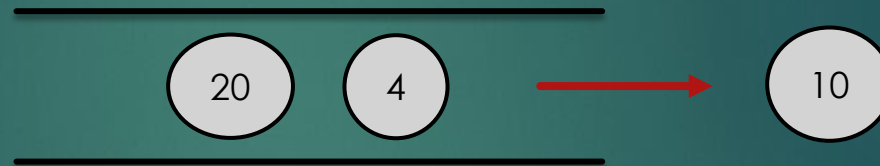
Deque **operacija**: uklanjanje elementa SA
POČETKA REDA // FIFO struktura

`queue.dequeue()`



Deque **operacija**: uklanjanje elementa SA
POČETKA REDA // FIFO struktura

`queue.dequeue()`



Deque **operacija**: uklanjanje elementa SA
POČETKA REDA // FIFO struktura

queue.dequeue()



Deque operacija: uklanjanje elementa SA
POČETKA REDA // FIFO struktura

`queue.dequeue()`



Deque **operacija**: uklanjanje elementa SA
POČETKA REDA // FIFO struktura



Osnovne operacije sa redovima

- Osnovne operacije za rad sa redovima su:
 - `enqueue(x, Q)`: ubacuje element `x` na kraj reda `Q`, ako je red pun dešava se prekoračenje `overflow` – vremenska složenost: $O(1)$
 - `dequeue(Q)`: uklanja i vraća element sa početka reda `Q`, ako je red prazan dešava se `underflow` uslov – vremenska složenost: $O(1)$
 - `front(Q)`: vraća element sa početka reda `Q`, ne menjajući ga – vremenska složenost: $O(1)$
 - `rear(Q)`: vraća element sa kraja reda `Q`, ne menjajući ga – vremenska složenost: $O(1)$
 - `make_null(Q)`: pretvara red `Q` u prazni red
 - `empty(Q)`: vraća `True` ako je red `Q` prazan, u suprotnom vraća `False`
- Pošto `Python` liste imaju potpunu funkcionalnost potrebnu za implementaciju redova, kreiranje reda i izvođenje operacija nad njim moguće je postići jednostavno, u par linija programskog kôda
- Drugi način je implementacija redova pomoću `klasa` i `objekata`

Osnovne operacije sa redovima



Operacija reda	Sadržaj reda	Povratna vrednost
D.enqueue(2)	[2]	-
D.enqueue(4)	[2, 4]	-
D.dequeue()	[4]	2
D.empty()	[4]	False
D.dequeue()	[]	4
D.empty()	[]	True
D.dequeue()	[]	Error: Emmpy queue
D.enqueue(7)	[7]	-
D.enqueue(5)	[7, 5]	-
D.dequeue()	[7, 5, 4]	7

Osnovne operacije sa redovima

- Red se može jednostavno implementirati pomoću niza (liste)
- Dodavanje elemenata u red preko `append()` a uklanjanje sa početka reda preko `pop(0)`

```
1  queue = []
2
3  # Enqueue metoda
4  queue.append('A')
5  queue.append('B')
6  queue.append('C')
7  print("Red: ", queue)
8
9  # Dequeue metoda
10 element = queue.pop(0)
11 print("Dequeue: ", element)
12
13 # Peek metoda
14 frontElement = queue[0]
15 print("Peek: ", frontElement)
16
17 # isEmpty metoda
18 isEmpty = not bool(queue)
19 print("Prazan red: ", isEmpty)
20
21 # Veličina reda
22 print("Veličina reda: ", len(queue))
```



```
Red: ['A', 'B', 'C']
Dequeue: A
Peek: B
Prazan red: False
Veličina reda: 2
```

Osnovne operacije sa redovima

- **Deque** (red sa dva kraja) je efikasniji za rad sa redovima od liste jer i `append()` i `popleft()` metode imaju vremensku složenost $O(1)$
- `popleft()` uklanja prvi element bez pomeranja, pa je **deque** dobar izbor za redove
- Koristi se `import` modula `deque` iz kolekcija: `from collections import deque`

```
1  from collections import deque
2  q = deque()
3
4  q.append('a')
5  q.append('b')
6  q.append('c')
7  print("Početni red:", q)
8
9  print("Elements uklonjeni iz reda:")
10 print(q.popleft())
11 print(q.popleft())
12 print(q.popleft())
13
14 print("Red je nakon uklanjanja elemenata:", q)
```



```
Početni red: deque(['a', 'b', 'c'])
Elements uklonjeni iz reda:
a
b
c
Red je nakon uklanjanja elemenata: deque([])
```

Implementacija reda preko niza

- Implementacija reda pomoću NIZA:
 - Kada se za implementaciju reda koristi niz, operacije dodavanja i brisanja elemenata iz reda se vrše osnovnim operacijama nad nizom, uz ograničenje nemogućnosti proširenja i skraćanja niza u zavisnosti od broja elemenata u redu
 - Za implementaciju reda koristi se niz konstante veličine koji mora biti dovoljan da se u njega smesti ukupan broj elemenata koji se dodaju u red. U svakom trenutku je neophodno znati indekse prvog i poslednjeg elementa u redu. Prilikom dodavanja elementa u red, povećava se indeks poslednjeg elementa i na tu poziciju u redu upisuje se novi element. U slučaju brisanja elementa iz reda, povećava se indeks prvog elementa u redu
 - Moguća je i implementacija koja bi zahtevala samo onoliku veličinu niza koliko je potrebno da se smeste elementi koji su u jednom trenutku u redu, ali takva realizacija zahteva stalno pomeranje preostalih elemenata prilikom brisanja prvog elementa iz reda

Implementacija reda preko niza

- Za implementaciju reda pomoću niza fiksne veličine koristimo:
 - Niz fiksne veličine `arr[]` za čuvanje elemenata reda
 - Promenljivu `size` za praćenje trenutnog broja elemenata u redu
 - Promenljivu `capacity` za čuvanje max. broja elemenata reda

```
1 class myQueue:
2     def __init__(self, capacity):
3
4         # Max. broj elemenata koji red može da čuva
5         self.capacity = capacity
6
7         # Niz za čuvanje elemenata reda
8         self.arr = [0] * capacity
9
10        # Tekući broj elemenata reda
11        self.size = 0
```

```
1 def enqueue(self, x):
2     if self.size == self.capacity:
3         print("Queue Overflow")
4         return
5
6     self.arr[self.size] = x
7     self.size += 1
8
```

```
1 def dequeue(self):
2     if self.size == 0:
3         print("Queue Underflow")
4         return
5
6     for i in range(1, self.size):
7         self.arr[i-1] = self.arr[i]
8     self.size -= 1
```

Implementacija reda preko niza

01
Step

enqueue(1): unet element u red, index je tekući size, red nije prazan, početno: front=0, size=0, capacity=4
arr[0]=1

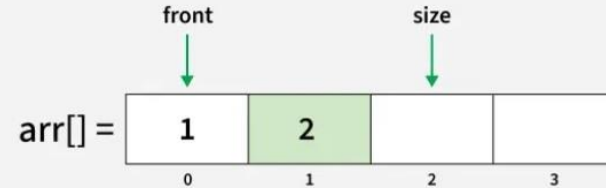
size = size + 1



02
Step

enqueue(2): unet element u red, index je tekući size, red nije prazan, front=0, size=1, capacity=4
arr[1]=2

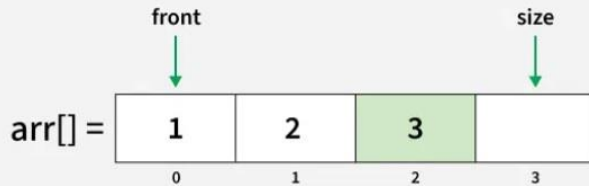
size = size + 1



03
Step

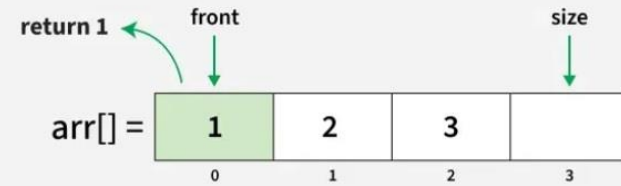
enqueue(3): unet sledeći element u red, index je tekući size, red nije prazan, front=0, size=2, capacity=4
arr[2]=3

size = size + 1



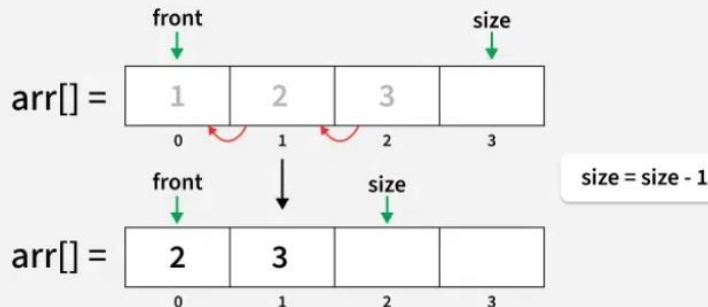
04
Step

getFront(): vraća front element reda, ako size nije 0, u suprotnom vraća -1



05
Step

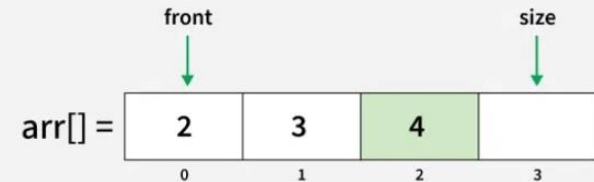
dequeue(): uklanja front element reda pomeranjem svih ostalih elemenata reda jedno mesto u levo, smanjujući size za 1



06
Step

enqueue(4): unosi se sledeći element u red, index je tekući size, red nije popunjen front=0, size=2, capacity=4
arr[2]=4

size = size + 1



Implementacija reda preko niza

```
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5
6 class Queue:
7     def __init__(self):
8         self.front = None
9         self.rear = None
10        self.length = 0
11
12    def enqueue(self, element):
13        new_node = Node(element)
14        if self.rear is None:
15            self.front = self.rear = new_node
16            self.length += 1
17            return
18        self.rear.next = new_node
19        self.rear = new_node
20        self.length += 1
21
22    def dequeue(self):
23        if self.isEmpty():
24            return "Red je prazan"
25        temp = self.front
26        self.front = temp.next
27        self.length -= 1
28        if self.front is None:
29            self.rear = None
30        return temp.data
```

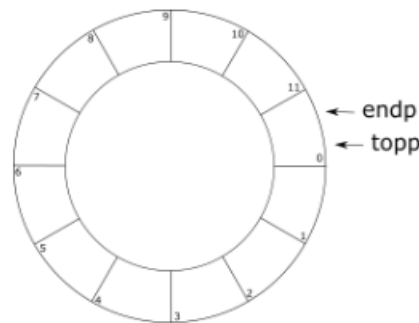
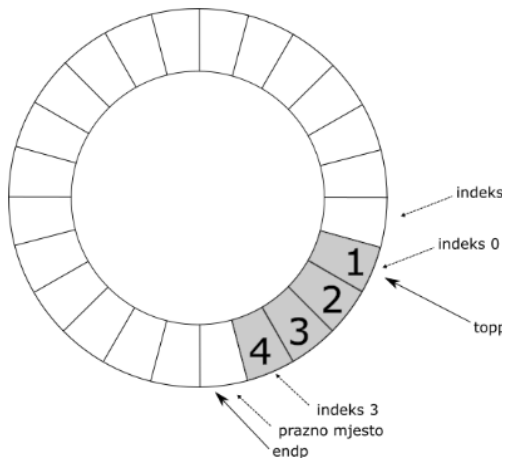
```
31
32    def peek(self):
33        if self.isEmpty():
34            return "Red je prazan"
35        return self.front.data
36
37    def isEmpty(self):
38        return self.length == 0
39
40    def size(self):
41        return self.length
42
43    def printQueue(self):
44        temp = self.front
45        while temp:
46            print(temp.data, end=" -> ")
47            temp = temp.next
48        print()
49
50    # Objekat reda Queue
51    myQueue = Queue()
52
53    myQueue.enqueue('A')
54    myQueue.enqueue('B')
55    myQueue.enqueue('C')
56    myQueue.enqueue('D')
57
58    print("Red: ", end="")
59    myQueue.printQueue()
60    print("Peek: ", myQueue.peek())
61    print("Dequeue: ", myQueue.dequeue())
62    print("Red nakon Dequeue: ", end="")
63    myQueue.printQueue()
64    print("Prazan red?: ", myQueue.isEmpty())
65    print("Veličina reda: ", myQueue.size())
```



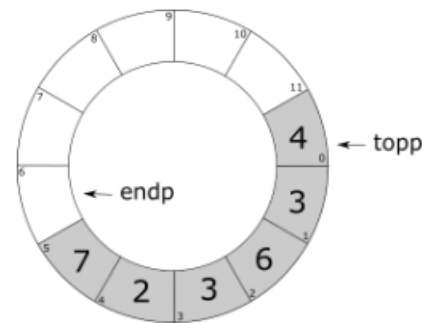
Red: A -> B -> C -> D ->
Peek: A
Dequeue: A
Red nakon Dequeue: B -> C -> D ->
Prazan red?: False
Veličina reda: 3

Implementacija reda kružnom listom

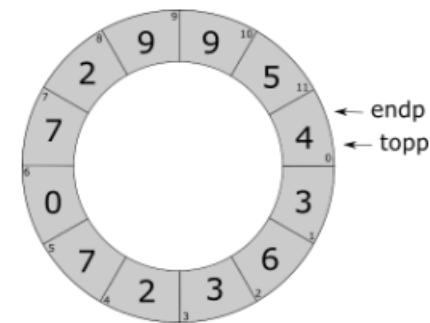
- Implementacija reda pomoću KRUŽNE (cirkularne) LISTE:
 - Kod kružne liste važi princip da kada dodajemo i brišemo elemente početak reda se pomiče
 - Korišćenjem kružne liste, obe operacije enqueue() i dequeue() se izvršavaju sa $O(1)$



(a) Prazni red



(b) Red s elementima



(c) Puni red

Implementacija reda kružnom listom

- Kod kružnih listi koristimo sledeće za implementaciju reda:
 - `arr[]`: niz za čuvanje elemenata reda
 - `capacity`: maksimalna veličina reda
 - `front`: index prvog elementa
 - `size`: tekući broj elemenata u redu

```
1 class myQueue:
2
3     def __init__(self, cap):
4         # Max. kapacitet reda
5         self.capacity = cap
6
7         # Fixed-size lista za elemente reda
8         self.arr = [0] * cap
9
10        # Index front elementa
11        self.front = 0
12
13        # Tekući broj elemenata reda
14        self.size = 0
```

```
1 # Insert elementa na rear poziciju
2 def enqueue(self, x):
3     if self.size == self.capacity:
4         print("Red je pun!")
5         return
6     rear = (self.front + self.size) % self.capacity
7     self.arr[rear] = x
8     self.size += 1
```

```
1 # Uklanjanje elementa sa front pozicije
2 def dequeue(self):
3     if self.size == 0:
4         print("Red je prazan!")
5         return -1
6     res = self.arr[self.front]
7     self.front = (self.front + 1) % self.capacity
8     self.size -= 1
9     return res
```

Implementacija reda kružnom listom

01
Step

enqueue: unos elementa 14
početno: front=0, size=0, capacity=4
 $\text{rear}=(\text{front}+\text{size})\% \text{capacity}=(0+0)\%4=0$
 $\text{arr}[0]=14$

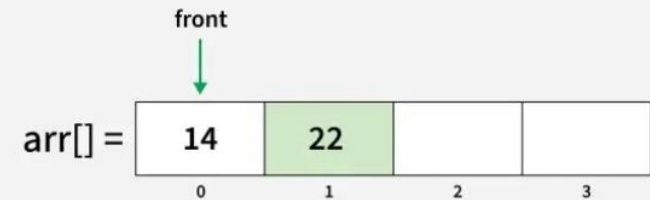
size = size + 1



02
Step

enqueue: unos elementa 22
front=0, size=1, capacity=4
 $\text{rear}=(\text{front}+\text{size})\% \text{capacity}=(0+1)\%4=1$
 $\text{arr}[1]=22$

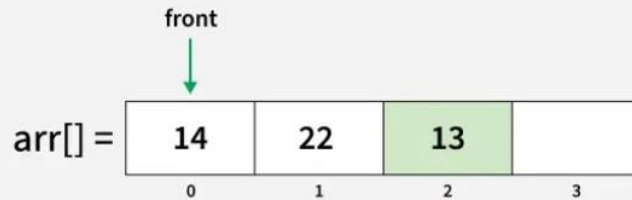
size = size + 1



03
Step

enqueue: unos elementa 13
front=0, size=2, capacity=4
 $\text{rear}=(\text{front}+\text{size})\% \text{capacity}=(0+2)\%4=2$
 $\text{arr}[2]=13$

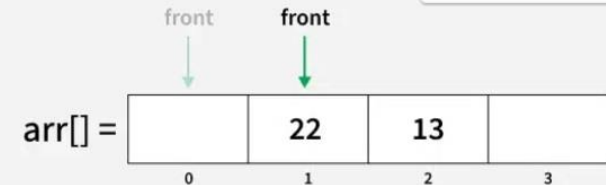
size = size + 1



04
Step

dequeue:
front=0, size=3, capacity=4

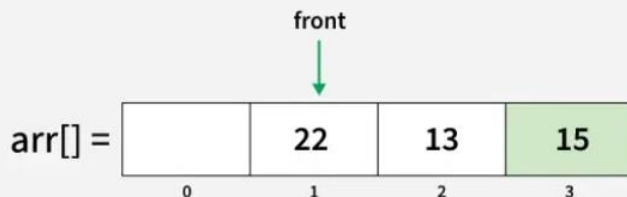
$\text{front} = (\text{front} + 1) \% \text{capacity}$
size = size - 1



05
Step

enqueue: unos elementa 15
front=1, size=2, capacity=4
 $\text{rear}=(\text{front}+\text{size})\% \text{capacity}=(1+2)\%4=3$
 $\text{arr}[3]=15$

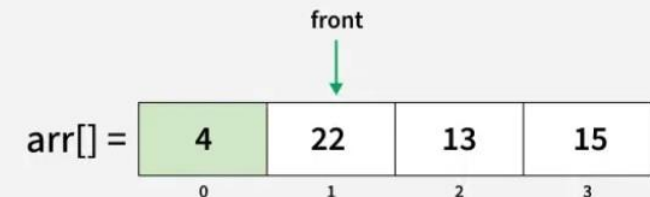
size = size + 1



06
Step

enqueue: unos elementa 4
front=1, size=3, capacity=4
 $\text{rear}=(\text{front}+\text{size})\% \text{capacity}=(1+3)\%4=0$
 $\text{arr}[0]=4$

size = size + 1



Implementacija reda kružnom listom

```
1 class myQueue:
2     def __init__(self, cap):
3         # fixed-size niz
4         self.arr = [0]*cap
5         # index front elementa
6         self.front = 0
7         # tekući broj elemenata
8         self.size = 0
9         # max. kapacitet reda
10        self.capacity = cap
11
12    # Insert element na rear
13    def enqueue(self, x):
14        if self.size == self.capacity:
15            print("Red je pun!")
16            return
17        rear = (self.front + self.size) % self.capacity
18        self.arr[rear] = x
19        self.size += 1
20
21    # Uklanjanje elementa sa front
22    def dequeue(self):
23        if self.size == 0:
24            print("Red je prazan!")
25            return -1
26        res = self.arr[self.front]
27        self.front = (self.front + 1) % self.capacity
28        self.size -= 1
29        return res
```

```
30
31    # Get the front element
32    def getFront(self):
33        if self.size == 0:
34            return -1
35        return self.arr[self.front]
36
37    # Get the rear element
38    def getRear(self):
39        if self.size == 0:
40            return -1
41        rear = (self.front + self.size - 1) % self.capacity
42        return self.arr[rear]
43
44    if __name__ == "__main__":
45        q = myQueue(5)
46        q.enqueue(10)
47        q.enqueue(20)
48        q.enqueue(30)
49        print(q.getFront(), q.getRear())
50        q.dequeue()
51        print(q.getFront(), q.getRear())
52        q.enqueue(40)
53        print(q.getFront(), q.getRear())
```

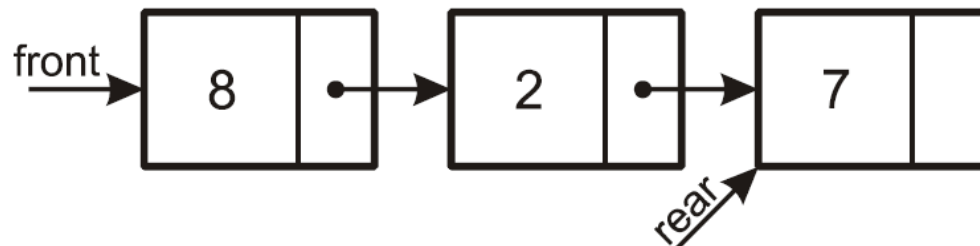
10 30

20 30

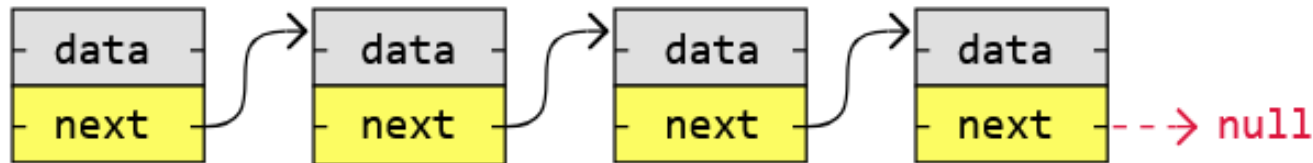
20 40

Implementacija reda pomoću povezane liste

- Implementacija reda pomoću POVEZANE LISTE:
 - Red se može efikasno implementirati korišćenjem povezanih listi tako što se novi element dodaje uvek na kraj liste
 - U slučaju brisanja elementa iz reda uklanjao bi se uvek prvi element u listi, odnosno onaj koji je prvi dodat
 - Na početku, lista je prazna, pa su i pokazivači na početak i kraj liste (**front** i **rear**) jednaki **NULL**
 - Metoda **insert()** kreira novi element i dodaje ga na kraj liste
 - Metoda **delete()** vraća vrednost prvog elementa u listi i pomera pokazivač **front** na sledeći element u listi
 - Na kraju se uništava element koji je bio na početku liste



Implementacija reda pomoću povezane liste



- Prednost korišćenja povezanih listi je što se čvorovi čuvaju gde ima slobodnog prostora u memoriji i čvorovi ne moraju biti čuvani uzastopno jedan iza drugog, kao što se elementi čuvaju u nizovima
- Još jedna prednost kod upotrebe povezanih listi je to što prilikom dodavanja ili uklanjanja čvorova, ostatak čvorova u listi ne mora biti pomeren
- Razlozi za korišćenje povezanih lista za implementaciju redova:
 - Red može dinamički da raste i smanjuje se, za razliku od nizova
 - Početni element reda može se ukloniti (staviti u red) bez potrebe za pomeranjem drugih elemenata u memoriji

Implementacija reda pomoću povezane liste

```
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5
6 class Queue:
7     def __init__(self):
8         self.front = None
9         self.rear = None
10        self.length = 0
11
12    def enqueue(self, element):
13        new_node = Node(element)
14        if self.rear is None:
15            self.front = self.rear = new_node
16            self.length += 1
17            return
18        self.rear.next = new_node
19        self.rear = new_node
20        self.length += 1
21
22    def dequeue(self):
23        if self.isEmpty():
24            return "Red je prazan"
25        temp = self.front
26        self.front = temp.next
27        self.length -= 1
28        if self.front is None:
29            self.rear = None
30        return temp.data
```

```
31
32    def peek(self):
33        if self.isEmpty():
34            return "Red je prazan"
35        return self.front.data
36
37    def isEmpty(self):
38        return self.length == 0
39
40    def size(self):
41        return self.length
42
43    def printQueue(self):
44        temp = self.front
45        while temp:
46            print(temp.data, end=" -> ")
47            temp = temp.next
48        print()
49
50    # Objekat reda Queue
51    myQueue = Queue()
52
53    myQueue.enqueue('A')
54    myQueue.enqueue('B')
55    myQueue.enqueue('C')
56    myQueue.enqueue('D')
57
58    print("Red: ", end="")
59    myQueue.printQueue()
60    print("Peek: ", myQueue.peek())
61    print("Dequeue: ", myQueue.dequeue())
62    print("Red nakon Dequeue: ", end="")
63    myQueue.printQueue()
64    print("Prazan red?: ", myQueue.isEmpty())
65    print("Veličina reda: ", myQueue.size())
```



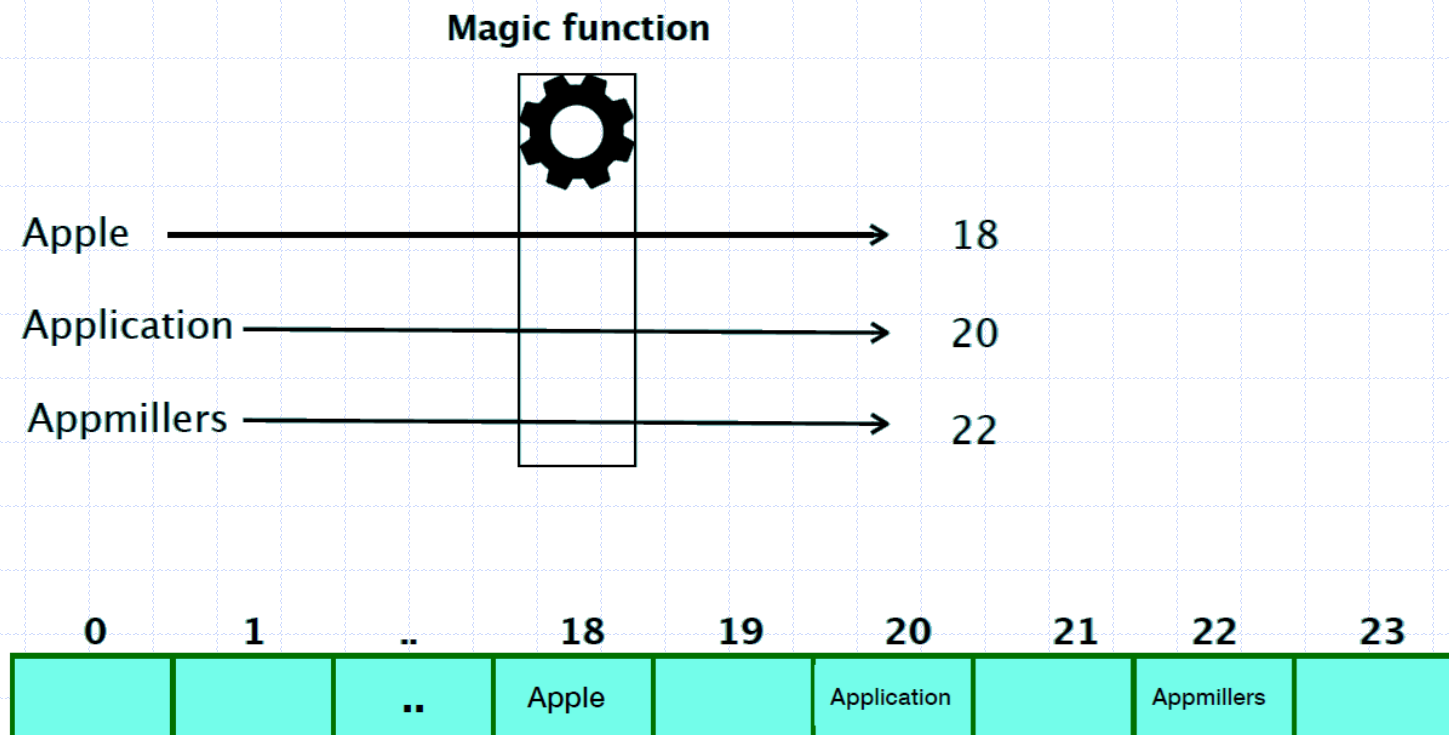
Red: A -> B -> C -> D ->
Peek: A
Dequeue: A
Red nakon Dequeue: B -> C -> D ->
Prazan red?: False
Veličina reda: 3

3. Struktura podataka: Heš tabele

Pojam heširanja

Pojam **heširanja**:

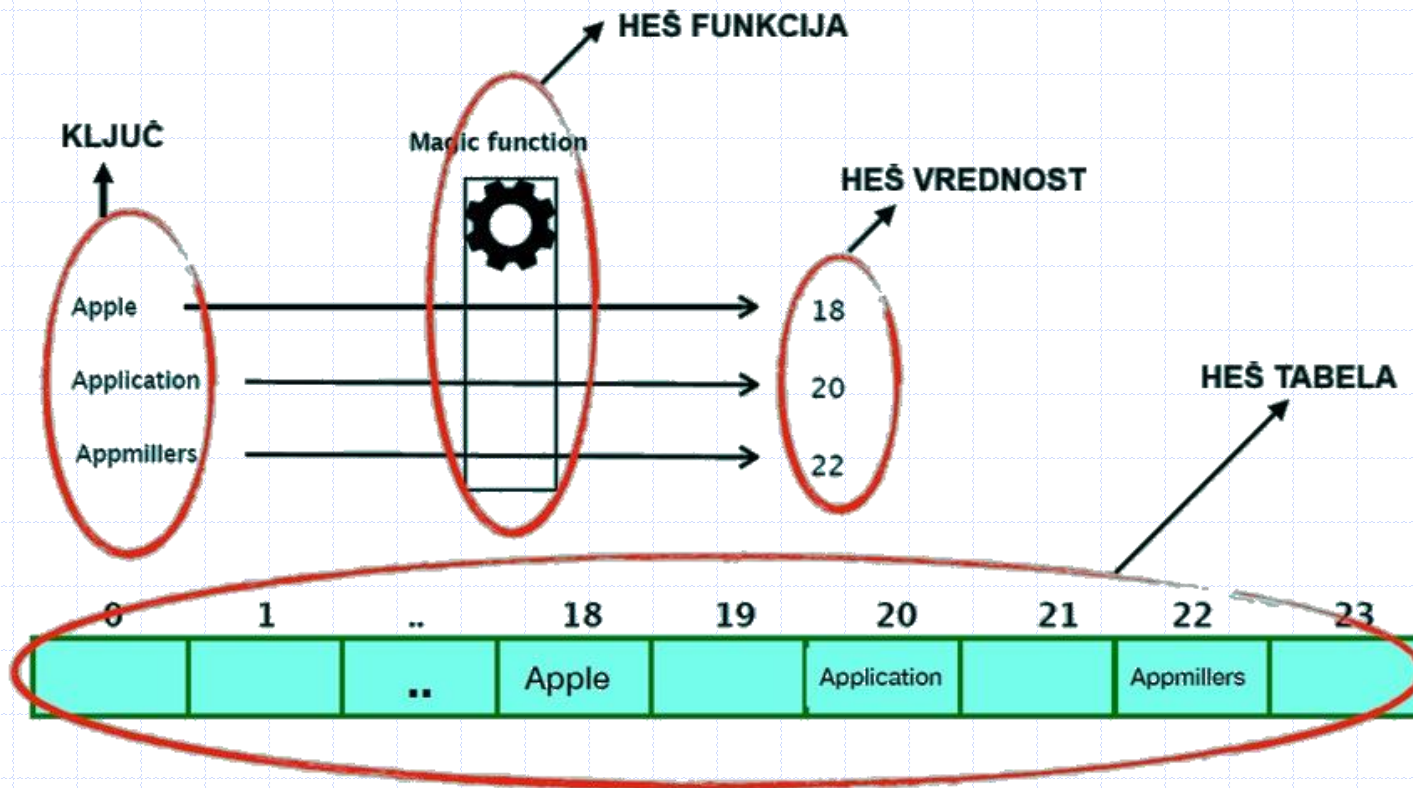
Heširanje je metoda sortiranja i indeksiranja podataka kod koje se indeksiranje velike količine podataka vrši ključevima koji se kreiraju matematičkom funkcijom



Heš tabele

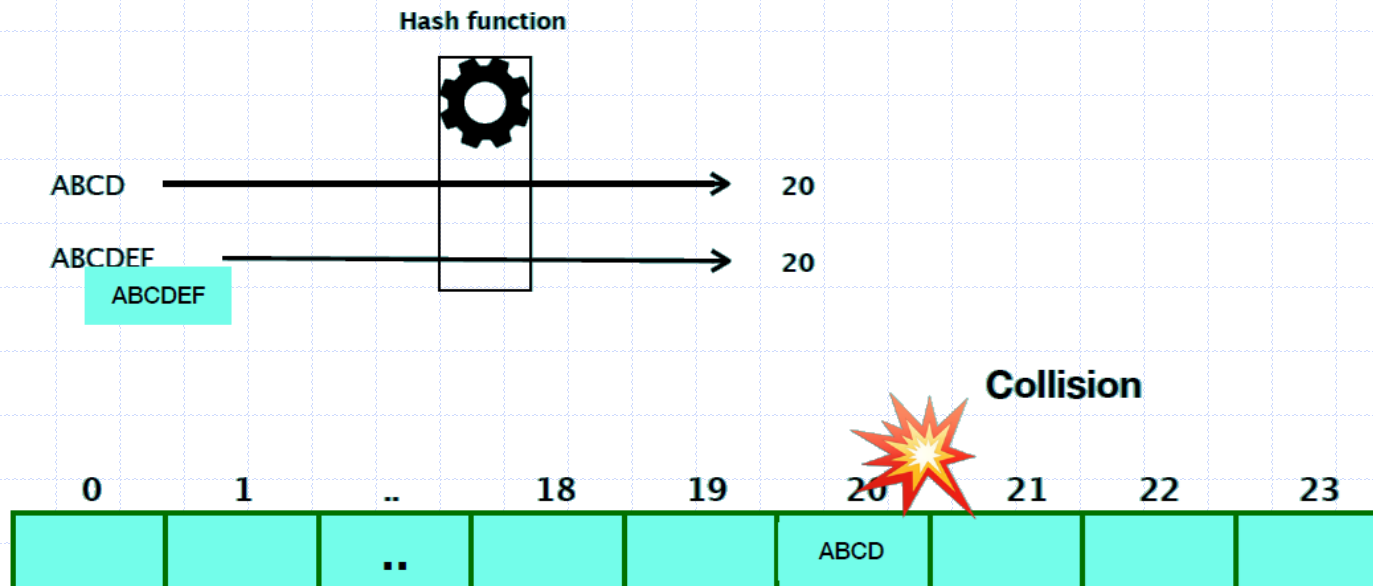
Heš tabela (hash table):

Struktura podataka koja implementira apstraktni tip podataka asocijativnog niza postupkom mapiranja parova ključ/vrednost (key/value) i koja je promenljiva (*mutable*)

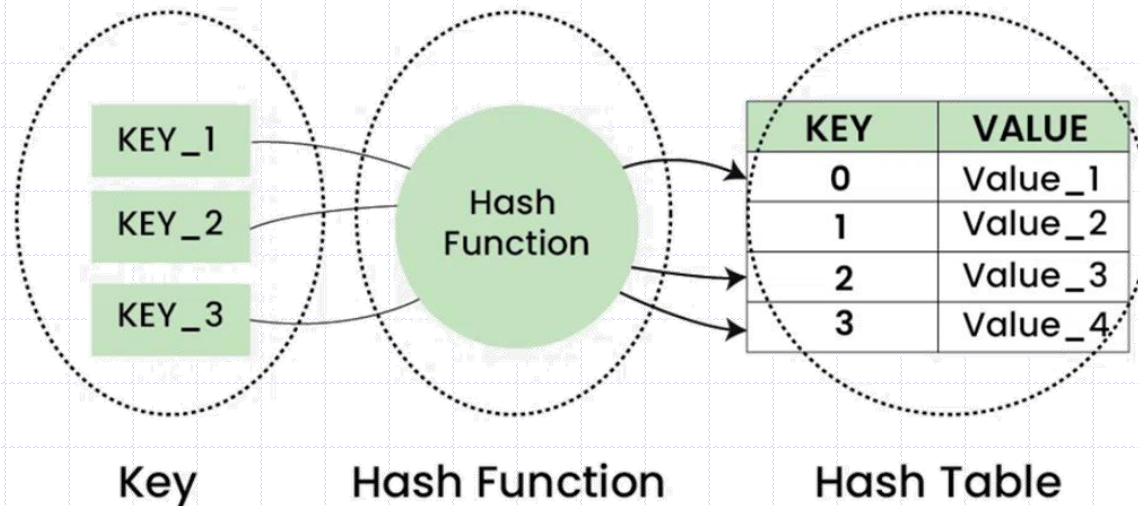


Komponente heširanja

- Ključ (key) predstavlja ulazni podatak
- Heš funkcija je funkcija za mapiranje podataka proizvoljne veličine na podatke fiksne veličine
- Heš vrednost je vrednost koju vraća heš funkcija
- Heš tabela je struktura podataka koja implementira niz kao tip podataka kroz mapiranje ključeva u vrednosti
- Kolizija (collision) nastaje kada dva različita ključa heš funkcije formiraju isti izlaz



Komponente heširanja



Tri komponente heširanja:

Ključ može biti string ili ceo broj koji je ulaz u heš funkciju, koja matematičkom funkcijom određuje indeks ili lokaciju za skladištenje elementa u strukturi podataka

Heš funkcija prihvata ulazni ključ i vraća indeks elementa u nizu koji se naziva heš tabela. Indeks predstavlja heš indeks

Heš tabela je struktura podataka koja mapira ključeve u vrednosti koristeći funkciju koja se naziva heš funkcija. Heš tabela čuva podatke u formi asocijativanog niz, gde svaka vrednost podataka ima svoj jedinstveni indeks

Heš tabele

- Heš tabela predstavljaju uopštenje standardnih nizova
- Heš tabela koristi niz čija je veličina proporcionalna broju ključeva koji su stvarno uskladišteni
- Za razliku od tabela da direktnim adresiranjem (*direct address table*) kod kojeg su ključevi numerički indeksi, heš tabele koriste indekse koji se računaju iz ključa
- Kada je broj uskladištenih ključeva mali u odnosu na ukupan broj mogućih ključeva, heš tabele predstavljaju alternativu za efektivno i direktno adresiranje niza
- Heš tabele su efikasne strukture podataka za implementaciju REČNIKA, u Python jeziku rečnik je jedna od osnovnih struktura
- Iako pretraga elemenata u heš tabeli može da traje kao i pretraga u povezanoj listi, u praksi se heširanje pokazalo kao dobro rešenje

Heš tabela

- Heš tabela se naziva i „tabela raspršivanja“
- Da bi se ostvarile podjednako dobre performanse kao kod indeksiranja niza, potrebno je u što većoj meri izbeći traženje ključa
- Osnovna ideja heširanja je da se umesto pretrage ključa njegov položaj u nizu izračunava na osnovi samog ključa, čime se izbegava nepotrebno pretraživanje velikog broja ključeva
- Heš tabela je struktura podataka koja može efikasno raditi sa brojem aktivnih (upotrebljenih) ključeva koji je puno manji od broja svih mogućih ključeva
- Ključ heš tabele ne mora isključivo biti numeričkog tipa i to se postiže upotrebom **heš funkcije** (*hash function*)

Heš tabele

- Iako skladištenje niza traje $O(1)$ vremena, pretraživanje u njemu traje najmanje $O(\log n)$ vremena
- Za veliki skup podataka to može biti sporo, i učiniti niz neefikasnim
- Potrebna je struktura koja može da skladišti podatke i pretražuje ih u konstantnom vremenu, tj. u $O(1)$
- Uvođenjem heš tabela, moguće je lako čuvati i preuzimati podatke u konstantnom vremenu $O(1)$
- **Prednosti heširanja** u strukturama podataka:
 - Heširanje je efikasan način za struktura ključ-vrednost
 - Heširanje omogućava brz pristup elementima sa konstantnom vremenskom složenošću
 - Operacije umetanja, brisanja i pretraživanja su veoma efikasne
 - Heširanje zahteva manje memorije jer dodeljuje fiksni prostor za skladištenje elemenata
 - Heširanje dobro funkcioniše sa velikim skupovima podataka, održavajući konstantno vreme pristupa
 - Heširanje je neophodno za bezbedno skladištenje podataka i verifikaciju integriteta

Heš funkcija

Lista = [11, 12, 13, 14, 15]

$$H(x) = [x \% 10]$$

	11%10	12%10	13%10	14%10	15%10
	↓	↓	↓	↓	↓
0	1	2	3	4	5
Hash Table	11	12	13	14	15

```
def mod(number, cellNumber):  
    return number % cellNumber
```

`mod(400, 24)` → 16

`mod(700, 24)` → 4

0	1	..	4	5	..	16	..	23
		..	700		..	400	..	

S obzirom da heš tabela u pozadini upotrebljava niz za smeštanje ključeva i vrednosti, ključ se mora transformirati u numeričku vrednost koja odgovara nekom indeksu tog niza

Primer heš funkcije:

Funkcija **modulo** (deljenje sa ostatkom), koja vraća ostatak pri celobrojnom deljenju

$$13\%10=3$$

Heš funkcija

ASCII function

```
def modASCII(string, cellNumber):  
    total = 0  
    for i in string:  
        total += ord(i)  
    return total % cellNumber
```

`modASCII("ABC", 24)` → 6

A → 65 $65+66+67 = 198$ | 24
B → 66 192 | 8
C → 67 6

ASCII Table

Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char
0	0	0		32	20	40	[space]	64	40	100	@
1	1	1	!	33	21	41	!	65	41	101	A
2	2	2	"	34	22	42	"	66	42	102	B
3	3	3	#	35	23	43	#	67	43	103	C
4	4	4	\$	36	24	44	\$	68	44	104	D
5	5	5	%	37	25	45	%	69	45	105	E
6	6	6	&	38	26	46	&	70	46	106	F
7	7	7	'	39	27	47	'	71	47	107	G
8	8	10	(	40	28	50	(	72	48	110	H
9	9	11	)	41	29	51	)	73	49	111	I
10	A	12	*	42	2A	52	*	74	4A	112	J
11	B	13	+	43	2B	53	+	75	4B	113	K
12	C	14	,	44	2C	54	,	76	4C	114	L
13	D	15	-	45	2D	55	-	77	4D	115	M
14	E	16	.	46	2E	56	.	78	4E	116	N
15	F	17	/	47	2F	57	/	79	4F	117	O
16	10	20	0	48	30	60	0	80	50	120	P
17	11	21	1	49	31	61	1	81	51	121	Q
18	12	22	2	50	32	62	2	82	52	122	R
19	13	23	3	51	33	63	3	83	53	123	S
20	14	24	4	52	34	64	4	84	54	124	T
21	15	25	5	53	35	65	5	85	55	125	U
22	16	26	6	54	36	66	6	86	56	126	V
23	17	27	7	55	37	67	7	87	57	127	W
24	18	30	8	56	38	70	8	88	58	130	X
25	19	31	9	57	39	71	9	89	59	131	Y
26	1A	32	:	58	3A	72	:	90	5A	132	Z
27	1B	33	;	59	3B	73	;	91	5B	133	[
28	1C	34	<	60	3C	74	<	92	5C	134	\
29	1D	35	=	61	3D	75	=	93	5D	135	]
30	1E	36	>	62	3E	76	>	94	5E	136	^
31	1F	37	?	63	3F	77	?	95	5F	137	_

0	1	..	6	7	..	16	..	23
		..	ABC		..		..	

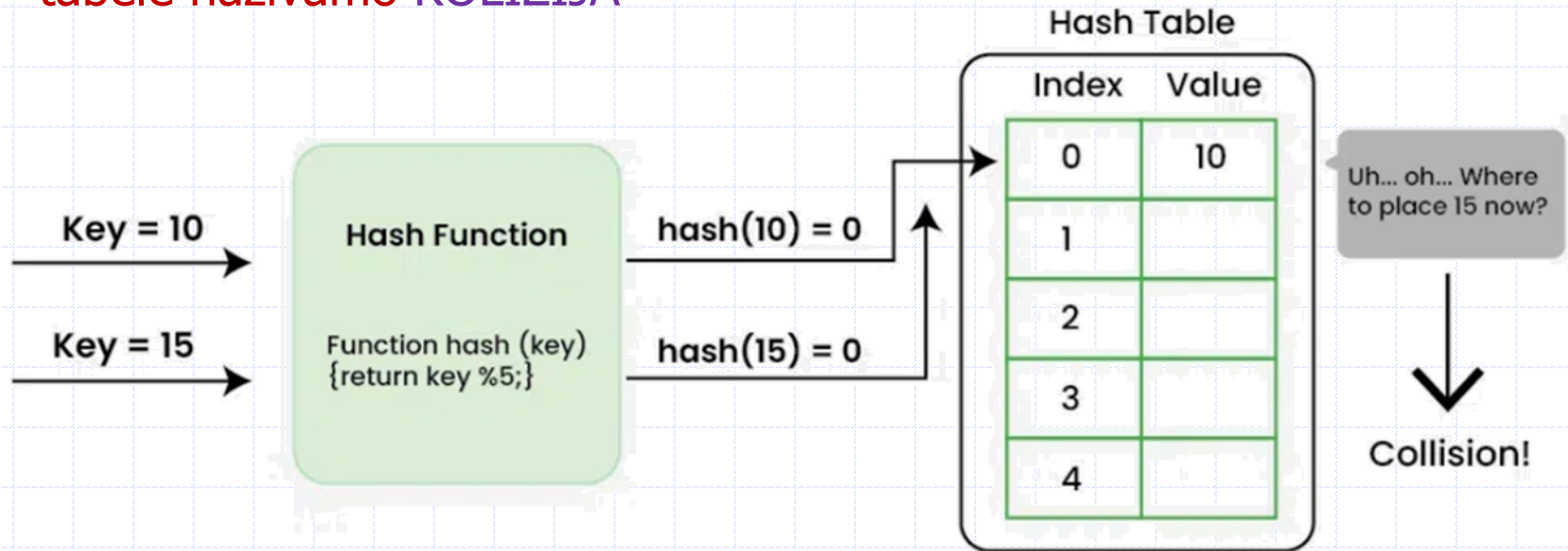
ord(i) – konvertuje karakter u ASCII vrednost
total+=ord(i) suma ASCII vrednosti
A+B+C=198%24=6

Dobra heš funkcija

- **Ravnomerna distribucija ključeva** heš tabele poželjna je karakteristika heš funkcije
- Jedan pojam koji je više od teorijskog značaja je nezavisna ravnomerna heš funkcija (*independent uniform hash function*)
- To je "idealna" heš funkcija kod koje je za svaki ključ k (iz skupa svih mogućih ključeva) svaka vrednost $h(k)$ iz domena $\{0, 1, \dots, m-1\}$ jednako verovatna ili $h(k)$ je element koji je slučajno i nezavisno od drugih elemenata odabran iz skupa elemenata $\{0, 1, \dots, m-1\}$
- **Dobra heš funkcija** aproksimativno zadovoljava pretpostavku jednostavnog, uniformnog heširanja: svaki ključ ima jednaku verovatnoću da se smesti u jednu od m ćelija, bez obzira gde su drugi ključevi heširani
- U praksi je obično nemoguće proveriti ovaj uslov, jer najčešće nije poznata verovatnoća distribucije ključeva

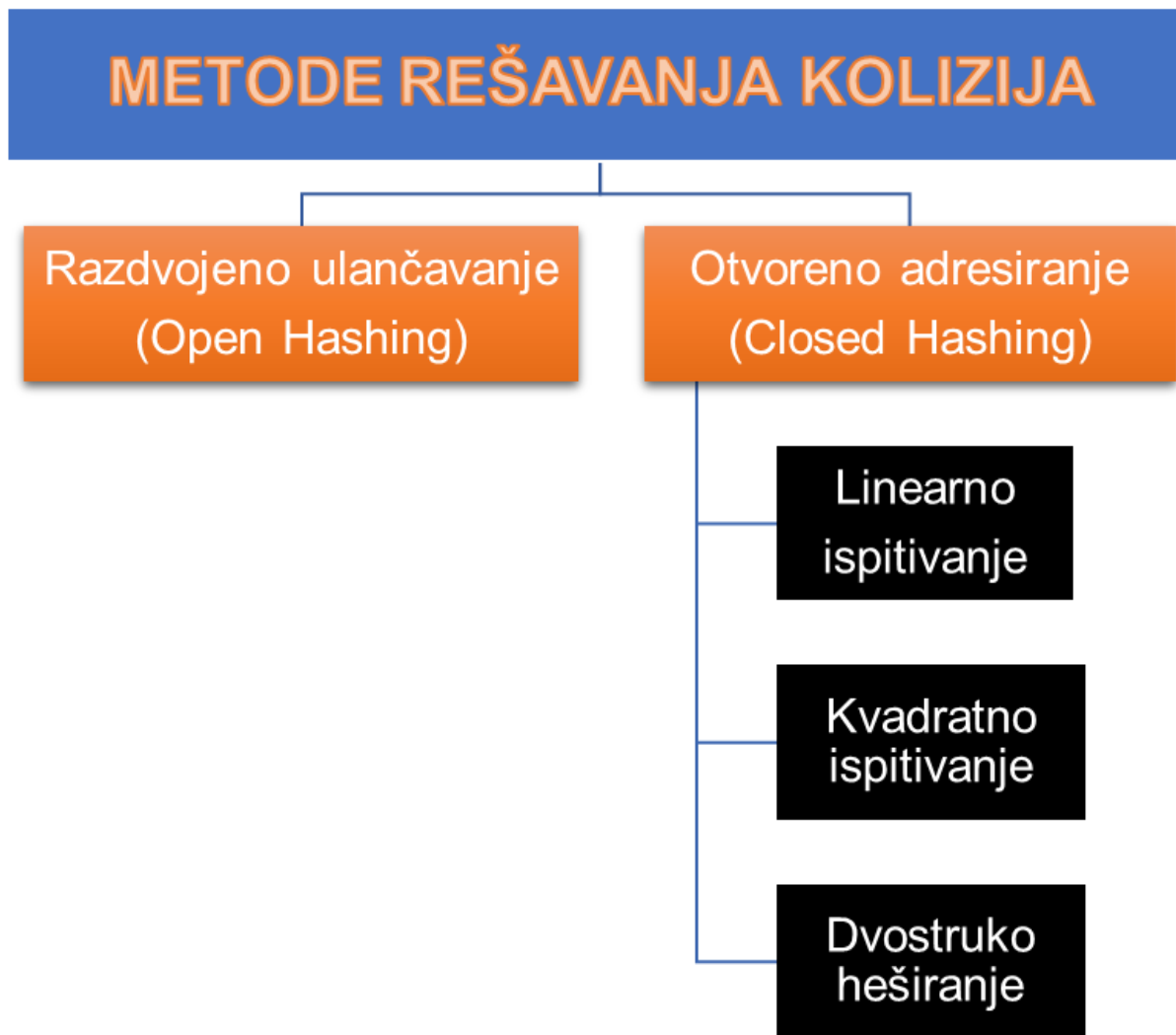
Kolizija heš tabela

- Pojavu kada se dva ili više ključeva moraju smestiti na isti indeks heš tabele nazivamo **KOLIZIJA**



- Kada je broj upotrebljenih ključeva veći od kapaciteta niza u koji se oni smeštaju, neki će indeksi morati da sadrže više ključeva pa postoji mogućnost da dva ključa imaju istu vrednost. Problem kada se novoumetnuti ključ mapira na već zauzet ključ rešava se neom od metoda za rukovanje kolizijama
- Ne postoji najbolja tehnika da se otkloni konflikt izazvan kolizijom

Metode rešavanja kolizija kod heširanja



Rešavanje kolizija ulančavanjem

- **Metoda ulančavanja** ili odvojenog niza je (*separate chaining*) vrši se heš funkcijom $h(x) = h_1(x)$
- Ulančavanje je tehnika rešavanja kolizija kod koje se svi ključevi koje treba smestiti na isti indeks niza heš tabele sačuvaju u povezanoj listi, zajedno s adresom pripadajuće vrednosti
- Ako treba pronaći zadati ključ, pretražuje se ta povezana lista, obično linearnim pretraživanjem
- Pristup ključu kod ovakve strukture podataka sastoji se u tome da se prvo odredi indeks na kojem se on nalazi upotrebom heš funkcije, a nakon toga unutar liste na tom se indeksu linearnim pretraživanjem pronađe se traženi ključ
- Za analizu performansi heš tabele koristi se vrednost **faktor opterećenja α** (load factor)
- Za heš tabelu veličine **m** i broj ključeva **n** faktor opterećenja iznosi **$\alpha = n/m$** i ta vrednost može biti veća, manja ili jednaka 1

Rešavanje kolizija ulančavanjem

Primer:

Data je heš funkcija, uneti elemente u heš tabelu metodom ulančavanja

Heš funkcija = $\text{key} \% 5$

Elementi = 12, 15, 22, 25, 37

Slot

0	
1	
2	
3	
4	

KORAK 1

prazna heš tabela sa opsegom vrednosti od 0 do 4 u skladu sa datom heš funkcijom



Slot

0	
1	
2	12
3	
4	

Korak 2

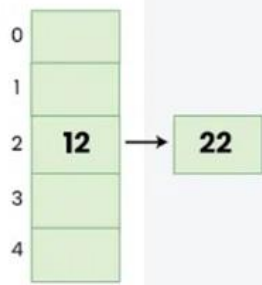
Prvi ključ koji treba uneti je 12, koji se mapira u slot 2

Rešavanje kolizija ulančavanjem

Heš funkcija = $\text{key} \% 5$

Elementi = 12, 15, 22, 25, 37

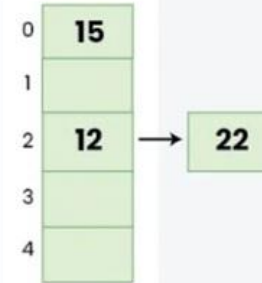
Slot



KORAK 3

Sledeći ključ je 22, mapira se u slot 2 jer je $22 \% 5 = 2$, u slotu 2 je već upisan ključ 12, formira se povezana lista za slot 2

Slot



KORAK 4

Sledeći ključ je 15 i on se mapira u slot 0 jer je $15 \% 5 = 0$

Slot



KORAK 5

Sledeći ključ je 25, mapira se u slot 0 jer je $25 \% 5 = 0$, u slotu 0 je već upisan ključ 15, za slot 0 se formira povezana lista

Rešavanje kolizija ulančavanjem

- Performanse heširanja mogu se proceniti pod pretpostavkom da je svaki ključ podjednako verovatno heširan u bilo koji slot tabele (jednostavno uniformno heširanje)
- Primer: Za milion vrednosti koje su raspoređene u dve liste od 500.000 vrednosti, pretraživanje neće biti efikasno kao kada bi bile raspoređene u 20.000 lista od 50 vrednosti, jer bi se za drugi slučaj pretraživao mnogo manji skup vrednosti. U najgorem slučaju svi su ključevi smešteni na isti indeks pa se pretraživanje svodi na linearno pretraživanje i vremenska složenost bi bila $\Theta(n)$
- Uz pretpostavku nezavisnog ravnomernog raspoređivanja, u tabeli u kojoj broj elemenata n nije veći od velčine tabele m ($\alpha = n/m = 1$) ključevi u tabeli će biti ravnomerno raspoređeni pa će time i liste imati mali broj elemenata
 - m** = broj slotova heš tabele
 - n** = broj ključeva za unos u heš tabelu
 - Faktor opterećenja **$\alpha = n/m$**
- Očekivano vreme pretraživanja ima složenost $O(1 + \alpha)$
- Očekivano vreme brisanja ključa ima složenost $O(1 + \alpha)$
- Pristup indeksu tabele i računanje heš funkcije ima složenost $O(1)$

Rešavanje kolizija otvorenim adresiranjem

- Otvoreno adresiranje (open addressing) je metoda rešavanja kolizija kod koje se svi elementi nalaze unutar same heš tabele
- Svaki indeks, sadrži ili element tabele ili NIL (oznaka praznog mesta), i faktor opterećenja α ne može biti veći od 1 kao kod ulančavanja
- Rešavanje kolizija: Pri dodavanju novog elementa prvo se odredi početni indeks u koji se on može sačuvati. Ako je taj indeks zauzet (kolizija) odredi se sledeći indeks koji ne mora biti idući po redu. Ovaj se postupak ponavlja dok se ne pronađe slobodan indeks
- Kod traženja ključa ispitujemo indekse tabele po istom ovom principu dok ne nađemo zadati ključ ili dok ne dođemo do praznog indeksa, što bi značilo da traženi ključ ne postoji u tabeli
- Za dodavanje novog ključa kod otvorenog adresiranja koristi se heš funkcija koja ima dva parametra – ključ i broj ispitivanja ili proba Ispitivanje (probe) je provera da li indeks u tabeli sadrži NIL
- Kod otvorenog adresiranja idealna funkcija raspršivanja ona koja zadovoljava svojstvo nezavisnog ravnomernog raspoređivanja permutacije (independent uniform permutation hashing). To znači da je ispitivanje za svaki ključ jedna jednako verovatna permutacija

Rešavanje kolizija linearnim ispitivanjem

- **Linearno ispitivanje** koristi princip dvostrukog heširanja, pri čemu je heš funkcija oblika

$$h(k,i) = (h'(k) + i) \bmod m$$

prvo se ispituje indeks $niz[h'(k)]$ zatim (ako je taj indeks zauzet) $niz[h'(k)+1]$, $niz[h'(k) + 2]$, ..., $niz[m - 1]$, $niz[0]$, $niz[1]$, ..., do indeksa $niz[h'(k) - 1]$

- Veličina tabele mora biti veća ili jednaka ukupnom broju ključeva i ovaj metod je poznat kao zatvoreno heširanje i zasniva se na ispitivanju
- Vrste ispitivanja:
 - **Insert(k)**: nastavi sa ispitivanjem dok se ne pronađe prazan slot, kada se pronađe ubaci **k**
 - **Search(k)**: Nastavi sa ispitivanjem dok ključ slota ne postane jednak **k** ili dok se ne dođe do praznog slota
 - **Delete(k)**: Ako se obriše ključ pretraga može da ne uspe slotovi obrisanih ključeva su posebno označeni kao „obrisani“, umetanjem može da ubaci stavka u obrisani slot, ali pretraga se ne zaustavlja na obrisanom slotu

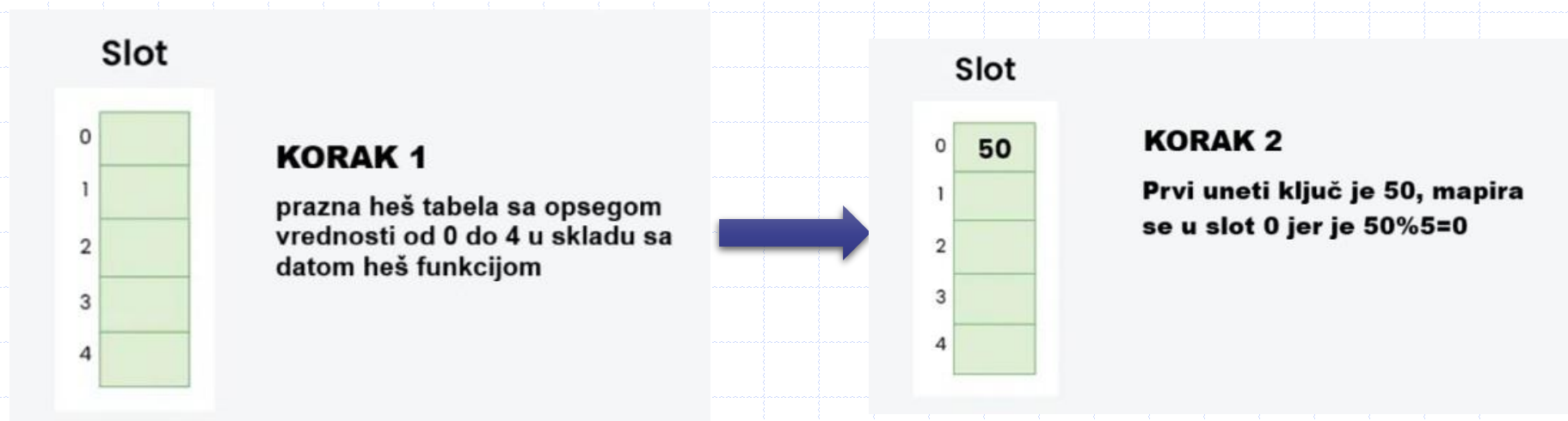
Rešavanje kolizija linearnim ispitivanjem

Primer:

Data je heš funkcija „key mod 5“, i sekvenca ključeva koje treba uneti u heš tabelu metodom linearnog ispitivanja

Heš funkcija = $\text{key} \% 5$

Ključevi = 50, 70, 75, 85, 93



Rešavanje kolizija linearnim ispitivanjem

Heš funkcija = $\text{key} \% 5$

Ključevi = 50, 70, 75, 85, 93

Slot

0	50
1	70
2	
3	
4	

KORAK 3

Sledeći ključ je 70, mapira se u slot 0, jer je $70 \% 5 = 0$, ali pošto je ključ 50 već u slotu 0, traži se sledeći prazan slot i koji se on ubacuje

Slot

0	50
1	70
2	76
3	
4	

KORAK 4

Sledeći ključ je 76, mapira se u slot 1 jer je $76 \% 5 = 1$, ključ 70 je već u slotu 1, traži se sledeći prazan slot za unos ključa 76

Slot

0	50
1	70
2	76
3	85
4	

KORAK 5

Sledeći ključ je 85, mapira se u slot 0 jer je $85 \% 5 = 0$, u slotu 0 je već ključ 50, traži se sledeći prazan slot, i ključ 85 se ubacuje u slot 3

Slot

0	50
1	70
2	76
3	85
4	93

KORAK 6

Sledeći ključ je 93, mapira se u slot 3 jer je $93 \% 5 = 3$, u slotu 3 je već ključ 85, traži se sledeći prazan slot i 93 se ubacuje u slot 4

Rešavanje kolizija linearnim ispitivanjem

- Kao i kod ulančavanja, performanse heširanja mogu se proceniti pod pretpostavkom da je svaki ključ istom verovatnoćom heširan u bilo koji slot tabele (jednostavno uniformno heširanje)
 - m = broj slotova heš tabele
 - n = broj ključeva koje treba uneti u heš tabelu
 - Faktor opterećenja $\alpha = n/m$ (< 1)
- Za neuspešno pretraživanje očekivani broj ispitivanja je najviše $1/(1 - \alpha)$
- Očekivani broj ispitivanja kod uspešnog pretraživanja je najviše $(1/\alpha)\ln(1/1-\alpha)$ pod pretpostavkom nezavisnog ravnomernog heširanja permutacije i da nema brisanja elemenata i da svaki ključ u tabeli ima jednaku verovatnoću pretraživanja
- Linearno ispitivanje ima najbolje performanse keširanja ali je problem grupisanje ili klasterovanje, i ova metoda je primenljiva zbog jednostavnosti izračunavanja

Rešavanje kolizija dvostrukim heširanjem

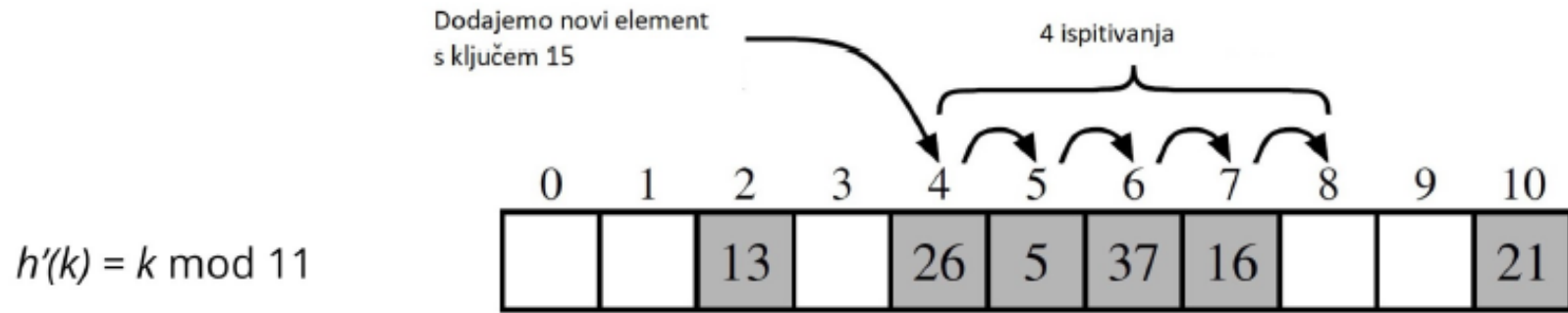
- Dvostruko heširanje smatra se jednom od najboljih metoda za otvoreno adresiranje jer daje permutacije koje dobro aproksimiraju svojstvo nezavisnog ravnomernog heširanja permutacije
- Ova metoda koristi heš funkciju u formi:

$$h(k,i) = (h_1(k) + ih_2(k)) \bmod m$$

gde su h_1 i h_2 pomoćne heš funkcije, početno ispitivanje je od indeksa $niz[h_1(k)]$ dok su naredna ispitivanja pomerena za $h_2(k)$ sa $\bmod m$ operacijom

- Dvostruko heširanje je tehnika koja smanjuje grupisanje na optimizovan način
- Inkrementi za sekvencu ispitivanja izračunavaju se upotrebom druge heš funkcije $hash_2(x)$, tako da se traži slot $i*hash_2(x)$ u i -toj rotaciji

Rešavanje kolizija dvostrukim heširanjem



$$h(79, 0) = 1$$

$$h(69, 0) = 4$$

$$h(72, 0) = 7$$

$$h(98, 0) = 7; \quad h(98, 1) = 5$$

$$h(14, 0) = 1; \quad h(14, 1) = 5; \quad h(14, 2) = 9$$

$$h(50, 0) = 11$$

- Za ključ **98** dolazi do kolizije jer je indeks **7** već zauzet pa se u idućem ispitivanju (ispitivanje=1) taj ključ sprema na indeks **5**
- Za ključ **14** dolazi do kolizije jer su indeksi **1** i **5** već zauzeti pa se u trećem ispitivanju (ispitivanje=2) on sprema na indeks **9**

Rešavanje kolizija dvostrukim heširanjem

Primer:

Uneti ključeve 27, 43, 692, 72 u heš tabelu veličine 7, gde je prva heš funkcija $h_1(k) = k \bmod 7$ a druga heš funkcija $h_2(k) = 1 + (k \bmod 5)$

Slot

0	
1	
2	
3	
4	
5	
6	

KORAK 1

Prazna heš tabela sa opsegom vrednosti od 0 do 6 u skladu sa datom heš funkcijom



Slot

0	
1	
2	
3	
4	
5	
6	27

KORAK 2

Prvi ključ koji se unosi je 27, mapira se u slot 6 jer je $27 \% 7 = 6$

Rešavanje kolizija dvostrukim heširanjem

Uneti ključeve 27, 43, 692, 72 u heš tabelu veličine 7, gde je prva heš funkcija $h_1(k)=k \bmod 7$ a druga heš funkcija $h_2(k) = 1+(k \bmod 5)$

Slot

0	
1	43
2	
3	
4	
5	
6	27

KORAK 3

Sledeći ključ je 43, koji se mapira u slot 1, jer je $43 \% 7 = 1$

Slot

0	
1	43
2	692
3	
4	
5	
6	27

KORAK 4

Sledeći ključ je 692, koji se mapira u slot 6 jer je $692 \% 7 = 6$, ali ovaj slot je već zauzet.

$$\begin{aligned} h_{\text{new}} &= [h_1(692) + i * (h_2(692))] \% 7 \\ &= [6 + 1 * (1 + 692 \% 5)] \% 7 \\ &= 9 \% 7 \\ &= 2 \end{aligned}$$

Pošto je 2 prazan slot, upisuje se ključ 692 u njega

Slot

0	
1	43
2	692
3	
4	
5	72
6	27

KORAK 5

Sledeći ključ je 72, mapira se u slot 2 jer je $72 \% 7 = 2$, ali je slot 2 već popunjen. Koristi se dvostruko heširanje:

$$\begin{aligned} h_{\text{new}} &= [h_1(72) + i * (h_2(72))] \% 7 \\ &= [2 + 1 * (1 + 72 \% 5)] \% 7 \\ &= 5 \% 7 \\ &= 5 \end{aligned}$$

Pošto je 5 prazan slot, upisuje se 72 u njega



Poređenje metoda za rešavanje kolizija

Razdvojeno ulančavanje	Otvoreno adresiranje
Jednostavnije za implementaciju	Zahteva složenija izračunavanja
Heš tabela se nikada ne popunjava, moguće je dodavanje elemenata u lanac	Heš tabela može biti potpuno popunjena
Manje zavisi od heš funkcije i faktora popunjavanja	Zahteva dodatnu pažnju kako bi se izbeglo grupisanje i faktor opterećenja
Koristi se ako se ne zna koliko i koliko često ključeva treba da bude uneto ili obrisano	Koristi se ako se zna koliko i koliko često ključeva treba da bude uneto ili obrisano
Keširanje nije efikasno jer se ključevi čuvaju u povezanoj listi	Keširanje je efikasno jer se sve čuva u istoj tabeli
Gubitak memorijskog prostora (neki delovi heš table se nikada ne koriste)	Slot se može koristiti čak i ako se ulaz ne mapira u njega
Koristi se dodatni prostor za ulančavanje	Ne postoje linkovi kod otvorenog adresiranja

Heširanje - primene

Password verification

Login Required

User:

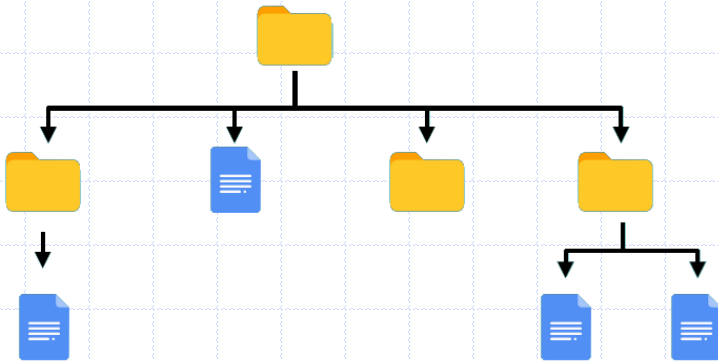
Password:

Login Cancel

Google Servers



File system : File path is mapped to physical location on disk



0	
1	/Documents/ Files/hashing.txt
2	
3	

Primer implementacije heš tabele

Primer: Heš tabela sa 7 slotova, ulazni ključevi {15,11,27,28}, heš funkcija $\text{hashIndex} = \text{key} \% \text{noOfSlots}$, uneti ključ na kraj liste, izbrisati čvor iz heš tabele

```
BUCKET_SIZE = 7
class Hash(object):
    def __init__(self, bucket):
        # Broj slotova
        self.__bucket = bucket
        # Heš tabela veličine slotova
        self.__table = [[] for _ in range(bucket)]

    # heš funkcija mapiranja vrednosti u ključ
    def hashFunction(self, key):
        return (key % self.__bucket)

    def insertItem(self, key):
        # dobijanje heš indeksa ključa
        index = self.hashFunction(key)
        self.__table[index].append(key)

    def deleteItem(self, key):
        # dobijanje heš indeksa ključa
        index = self.hashFunction(key)

        # Provera da li je ključ u heš tabeli
        if key not in self.__table[index]:
            return
```

```
# delete the key from hash table
self.__table[index].remove(key)

# funkcija za prikaz heš tbele
def displayHash(self):
    for i in range(self.__bucket):
        print("[%d]" % i, end='')
        for x in self.__table[i]:
            print(" --> %d" % x, end='')
        print()

if __name__ == "__main__":
    # niz sa ključevima koji se mapiraju
    a = [15, 11, 27, 8, 12]

    # Kreiranje a praznih slotova sa BUCKET_SIZE
    h = Hash(BUCKET_SIZE)

    # insert ključeva u heš tabelu
    for x in a:
        h.insertItem(x)

    # delete 12 iz heš tabele
    h.deleteItem(x)

    # Prikaz heš tabele
    h.displayHash()
```

Output

```
0
1 --> 15 --> 8
2
3
4 --> 11
5
6 --> 27
```

Python - rečníci

- Rečník se kreira na više načina:

```
>>> empty_dict={} # prazan rečník
```

```
>>> dct = {'key1':'value1', 'key2':'value2', 'key3':'value3',  
          'key4':'value4'}
```

- Dužina rečníka:

```
>>> dct={'key1':'value1', 'key2':'value2', 'key3':'value3',  
        'key4':'value4'}
```

```
>>> print(len(dct)) # 4
```

- Pristup elementima rečníka:

```
dct = {'key1':'value1', 'key2':'value2', 'key3':'value3',  
       'key4':'value4'}
```

```
print(dct['key1']) # value1
```

```
print(dct['key4']) # value4
```

- Izmena elementa rečníka:

```
dct = {'key1':'value1', 'key2':'value2', 'key3':'value3',  
       'key4':'value4'}
```

```
dct['key1'] = 'value-one'
```

Python - rečníci

- Poređenje rečnika:

```
>>> dict1={'Ime':'Ana','Godine':25}
>>> dict2={'Ime':'Anka','Godine':22}
>>> dict1={'Ime':'Ana','Godine':25}
>>> print (dict1==dict2) #False
>>> print (dict1==dict3) #True
```

- Kopiranje rečnika:

```
>>> dict1={'Ime':'Ana','Godine':25}
>>> dict4=dict1.copy()
>>> print(dict4) #{'Godine':25,'Ime':'Ana'}
```

- Prikaz svih elemenata rečnika u formi ključ/vrednost:

```
>>> dict1={'Ime':'Ana','Godine':25}
>>> print("Vrednosti: %s" % dict.items())
#Vrednosti: [('Godine',25),('Ime','Ana')]
```

Python rečníci - metode

Metoda	Opis
<code>A.clear()</code>	Uklanjanje svih elemenata iz rečnika
<code>A.copy()</code>	Vraća plitku (<i>shallow</i>) kopiju rečnika.
<code>dict.fromkeys(seq[, v])</code>	Vraća novi rečnik sa ključevima iz <i>seq</i> i vrednostima <i>v</i> (ako vrednost nije data onda <i>None</i>)
<code>A.get(key[, d])</code>	Vraća vrednost za ključ <i>key</i> . Ako ne postoji , onda vraća <i>d</i> , a ako nije dato <i>d</i> vraća <i>None</i>
<code>A.items()</code>	Vraća <i>view</i> svih elemenata u rečniku u obliku (<i>key, value</i>).
<code>A.keys()</code>	Vraća <i>view</i> svih ključeva u rečniku
<code>A.pop(key[, d])</code>	Uklanja element sa ključem <i>key</i> i vraća njegovu vrednost, ili <i>d</i> ako nije pronađen, a ako je bez <i>d</i> javlja <i>KeyError</i> .
<code>A.popitem()</code>	Uklanja i vraća jedan proizvoljni element (<i>key, value</i>). Javlja <i>KeyError</i> za prazan rečnik

Python rečníci - metode

Metoda	Opis
A.setdefault (key[, d])	Ako je <i>key</i> u rečniku, vraća vrednost. Ako nije, dodaje <i>key</i> sa vrednošću <i>d</i> i vraća tu vrednost
A.update ([other])	Ažurira rečnik parovima <i>key/value</i> iz <i>other</i> , ako neki ključ postoji, upisuje preko postojećeg
A.values ()	Vraća <i>view</i> svih vrednosti u rečniku
A.all ()	Vraća True ako su svi ključevi true, ili ako je rečnik prazan
A.any ()	Vraća True ako je bilo koji ključ true. Za prazan vraća False.
len (A)	Vraća broj elemenata u rečniku
sorted (A)	Vraća novu,sortiranu listu ključeva u rečniku

